arXiv:2004.00518v2 [cs.DC] 2 Apr 2020

Scheduling Parallel-Task Jobs Subject to Packing and Placement Constraints

Mehrnoosh Shafiee and Javad Ghaderi
Department of Electrical Engineering
Columbia University

ABSTRACT

Motivated by modern parallel computing applications, we consider the problem of scheduling parallel-task jobs with heterogeneous resource requirements in a cluster of machines. Each job consists of a set of tasks that can be processed in parallel, however, the job is considered completed only when all its tasks finish their processing, which we refer to as "*synchronization*" constraint. Further, assignment of tasks to machines is subject to "*placement*" constraints, i.e., each task can be processed only on a subset of machines, and processing times can also be machine dependent. Once a task is scheduled on a machine, it requires a certain amount of resource from that machine for the duration of its processing. A machine can process ("*pack*") multiple tasks at the same time, however the cumulative resource requirement of the tasks should not exceed the machine's capacity.

Our objective is to minimize the weighted average of the jobs' completion times. The problem, subject to synchronization, packing and placement constraints, is NP-hard, and prior theoretical results only concern much simpler models. For the case that migration of tasks among the placement-feasible machines is allowed, we propose a preemptive algorithm with an approximation ratio of $(6+\epsilon)$. In the special case that only one machine can process each task, we design an algorithm with improved approximation ratio of 4. Finally, in the case that migrations (and preemptions) are not allowed, we design an algorithm with an approximation ratio of 24. Our algorithms use a combination of linear program relaxation and greedy packing techniques. We present extensive simulation results, using a real traffic trace, that demonstrate that our algorithms yield significant gains over the prior approaches.

KEYWORDS

Scheduling Algorithms, Approximation Algorithms, Data-Parallel Computing, Datacenters

1 INTRODUCTION

Modern parallel computing frameworks (e.g. Hadoop [10], Spark [12]) have enabled large-scale data processing in computing clusters. In such frameworks, the data is typically distributed across a cluster of machines and is processed in multiple stages. In each stage, a set of tasks are executed on the machines, and once all the tasks in the stage finish their processing, the job is finished or moved to the next stage. For example, in MapReduce [9], in the map stage, each map task performs local computation on a data block in a machine and writes the intermediate data to the disk. In the reduce stage, each reduce task pulls intermediate data from different maps, merges them, and computes its output. While the reduce tasks can start pulling data as map tasks finish, the actual computation by the reduce tasks can only start once all the map tasks are done and their data pieces are received. Further, the job is not completed unless all the reduce tasks finish. Similarly, in Spark [12, 45], the computation is done in multiple stages. The tasks in a stage can run in parallel, however, the next stage cannot start unless the tasks in the preceding stage(s) are all completed.

We refer to such constraints as *synchronization* constraints, i.e., a stage is considered completed only when all its tasks finish their processing. Such synchronizations could have a significant impact on the jobs' latency in parallel computing clusters [3, 6, 22, 44, 45]. Intuitively, an efficient scheduler should complete all the (inhomogeneous) tasks of a stage more or less around the same time, while prioritizing the stages of different jobs in an order that minimizes the overall latency in the system.

Another main feature of parallel computing clusters is that jobs can have diverse tasks and processing requirements. This has been further amplified by the increasing complexity of workloads, i.e., from traditional batch jobs, to queries, graph processing, streaming, and machine learning jobs, that all need to share the same cluster. The cluster manager (*scheduler*) serves the tasks of various jobs by reserving their requested resources (e.g. CPU, memory, etc.). For example, in Hadoop [11], the resource manager reserves the tasks' resource requirements by launching "*containers*" in machines. Each container reserves required resources for processing of a task. To improve the overall latency, we therefore need a scheduler that *packs* as many tasks as possible in the machines, while retaining their resource requirements.

In practice, there are further placement constraints for processing tasks on machines. For example, each task is preferred to be scheduled on one of the machines that has its required data block [2, 9] (a.k.a. data locality), otherwise processing can slow down due to data transfer. The data block might be stored in multiple machines for robustness and failure considerations. However, if all these machines are highly loaded, the scheduler might actually need to schedule the task in a less loaded machine that does not contain the data.

Despite the vast literature on scheduling algorithms, the theoretical results are mainly based on simple models where each machine processes one task at a time, each job is a single task, or tasks can be processed on any machine arbitrarily (see Related Work in Section 1.1). The objective of this paper is to design scheduling algorithms, with *theoretical guarantees*, under the modern features of data-parallel computing clusters, namely,

- *packing constraint*: each machine is capable of processing multiple tasks at a time subject to its capacity.
- *synchronization constraint*: tasks that belong to the same stage (job) have a collective completion time which is determined by the slowest task in the collection.

- *placement constraint*: task's processing time is machine-dependent and task is typically preferred to be processed on a subset of machines (e.g. where its input data block is located).

For simplicity, we consider one dimension for task resource requirement (e.g. memory). While task resource requirements are in general multi-dimensional (CPU, memory, etc.), it has been observed that memory is typically the bottleneck resource [11, 28].

Note that the scheduler can only make scheduling decisions for the stages that have been released from various jobs up to that point (i.e., those that their preceding stages have been completed). In our model, we use the terms stage and job interchangeability. Our objective is to minimize the weighted sum of completion times of existing jobs (stages) in the system, where weights can encode different priorities for the jobs (stages). Clearly minimization of the average completion time is a special case of this problem with equal weights. We consider both preemptive and non-preemptive scheduling. In a *non-preemptive* schedule, a task cannot be preempted (and hence cannot be migrated among machines) once it starts processing on a machine until it is completed. In a *preemptive* schedule, a task may be preempted and resumed later in the schedule, and we further consider two cases depending on whether migration of a task among machines is allowed or not.

1.1 Related Work

Default cluster schedulers in Hadoop [13, 14, 43] focus primarily on fairness and data locality. Such schedulers can make poor scheduling decisions by not packing tasks well together, or having a task running long without enough parallelism with other tasks in the same job. Several cluster schedulers have been proposed to improve job completion times, e.g. [18, 19, 21, 25, 31, 36, 39, 40, 42]. However, they either do not consider all aspects of packing, synchronization, and data locality, or use heuristics which are not necessarily efficient.

We highlight four relevant papers [18, 39, 40, 42] here. Tetris [18] is a scheduler that assigns scores to tasks based on Best-Fit bin packing and Shortest-Remaining-Time-First (SRPF) heuristic, and gives priority to tasks with higher scores. The data locality is encoded in scores by imposing a remote penalty to penalize use of remote resources. Borg [39] packs multiple tasks of jobs in machines from high to low priority, modulated by a round-robin scheme within a priority to ensure fairness across jobs. The scheduler considers data locality by assigning tasks to machines that already have the necessary data stored. The papers [40] and [42] focus on single-task jobs and study the mean delay of tasks under a stochastic model where if a task is scheduled on one of the remote servers that do not have the input data, its average processing time will be larger, by a multiplicative factor, compared to the case that it is processed on a local server that contains the data. They propose algorithms based on Join-the-Shortest-Queue and Max-Weight (JSQ-MW) to incorporate data locality in load balancing. This model is generalized in [42] to more levels of data locality. However, these models do not consider any task packing in servers or synchronization issue among multiple tasks of the same job.

From a theoretical perspective, our problem of scheduling parallel-task jobs with synchronization, packing, and placement constraints, can be seen as a generalization of the concurrent open shop (COS) problem [1]. Unlike COS, where each machine processes one task at a time and each task can be processed on a specific machine, in our model a machine can process (pack) multiple tasks simultaneously subject to its capacity, and there are further task placement constraints for assigning tasks to machines. Minimizing the weighted sum of completion times in COS, is known to be APX-hard [17], with several 2-approximation algorithms in [4, 7, 17, 24, 27, 33]. There is also a line of research on the parallel tasks scheduling (PTS) problem [16]. In PTS, each job is only a single task that requires a certain amount of resource for its processing time, and can be served by any machine subject to its capacity. This differs from our model where each job has multiple tasks, each task can be served by a set of machines, and the job's completion time is determined by its last task. Minimizing the weighted sum of completion times in the PTS is also NP-complete in the strong sense [5]. In the case of a single machine, the non-preemptive algorithm in [37] can achieve approximation ratio of 7.11, and the preemptive algorithm in [37], called *PSRS*, can achieve approximation ratio of 2.37. In the case of multiple machines, there is only one result in the literature which is a 14.85-approximation non-preemptive algorithm [32].

We emphasize that our setting of parallel-task jobs, subject to synchronization, packing, and placement constraints, is significantly more challenging than the COS and PTS problems, and algorithms from these problems *cannot* be applied to our setting. To the best of our knowledge, this is the first paper that provides constant-approximation algorithms for this problem subject to synchronization, packing, and placement constraints,

1.2 Main Contributions

We briefly summarize our main results and describe our techniques below. We propose scheduling algorithms for three cases:

- **Task Migration Allowed.** When migration is allowed, a task might be preempted several times and resume possibly on a different machine within its placement-feasible set. Our algorithm in this case is based on greedy scheduling of task fractions (fraction of processing time of each task) on each machine, subject to capacity and placement constraints. The task fractions are found by solving a relaxed linear program (LP), which divides the time horizon into geometrically-increasing time intervals, and uses *interval-indexed variables* to indicate what fraction of each task is served at which interval on each machine. We show that our scheduling algorithm has an approximation ratio better than $(6 + \epsilon)$, for any $\epsilon > 0$.
- **Task Migration Not Allowed.** When migration is not allowed, the schedule can be non-preemptive, or preemptive while all preemptions occur on the same machine. In this case, our algorithm is based on mapping tasks to proper time intervals on the machines. We utilize the interval-indexed variables to form a relaxed LP. We then utilize the LP's optimal solution to construct a *weighted bipartite graph* representing tasks on one side and machine-intervals on the other side, and fractions of tasks completed in machine-intervals as weighted edges. We then use an integral matching in this graph to construct a mapping of tasks to machine-intervals. Finally, the tasks mapped to intervals of the same machine are packed in order and non-preemptively by using a greedy policy. We prove that this non-preemptive

algorithm has an approximation ratio better than 24. Further, we show that the algorithm's solution is also a 24-approximation for the case that preemption on the same machine is allowed.

- **Preemption and Single-Machine Placement Set.** When preemption is allowed, and there is a specific machine for each task, we propose an algorithm with an improved approximation ratio of 4. The algorithm first finds a proper ordering of jobs, by solving a relaxed LP of our scheduling problem. Then, for each machine, it lists its tasks, with respect to the obtained ordering of jobs, and apply a simple greedy policy to pack tasks in the machine subject to its capacity. The methods of LP relaxation and list scheduling have been used in scheduling literature; however, the application and analysis of such techniques in presence of packing, placement, and synchronization is very different.
- **Empirical Evaluations.** We evaluate the performance of our preemptive and non-preemptive algorithms compared with the prior approaches using a Google traffic trace [41]. We also present online versions of our algorithms that are suitable for handling dynamic job arrivals. Our 4−approximation preemptive algorithm outperforms PSRS [37] and Tetris [18] by up to 69% and 79%, respectively, when jobs' weights are determined using their priority information in the data set. Further, our non-preemptive algorithm outperforms JSQ-MW [40] and Tetris [18] by up to 81% and 175%, respectively, under the same placement constraints.

2 FORMAL PROBLEM STATEMENT

Cluster and Job Model. Consider a collection of machines $\mathcal{M} = \{1, ..., M\}$, where machine i has capacity $m_i > 0$ on its available resource. We use $\mathcal{J} = \{1, ..., N\}$ to denote the set of existing jobs (stages) in the system that need to be served by the machines. Each job $j \in \mathcal{J}$ consists of a set of tasks $\mathcal{K}_j$, where we use (k, j) to denote task k of job j, $k \in \mathcal{K}_j$. Task (k, j) requires a specific amount a_{kj} of resource for the duration of its processing.

Task Processing and Placement Constraint. Each task (k, j) can be processed on a machine from a specific set of machines $\mathcal{M}_{kj} \subseteq \mathcal{M}$. We refer to $\mathcal{M}_{kj}$ as the *placement set* of task (k, j). For generality, we let p^i_{kj} denote the processing time of task (k, j) on machine $i \in \mathcal{M}_{kj}$. Such placement constraints can model data locality. For example, we can set $\mathcal{M}_{kj}$ to be the set of machines that have task (k, j)'s data, and $p^i_{kj} = p_{kj}$, $i \in \mathcal{M}_{kj}$. Or, we can consider $\mathcal{M}_{kj}$ to be as large as $\mathcal{M}$, and incorporate the data transfer cost as a penalty in the processing time on machines that do not have the task's data.

Throughout the paper, we refer to a_{kj} as size or resource requirement of task (k, j), and to p^i_{kj} as its length, duration, or processing time on machine i. We also define the volume of task (k, j) on machine i as $v^i_{kj} = a_{kj}p^i_{kj}$. Without loss of generality, we assume processing times are nonnegative integers and duration of the smallest task is at least one. This can be done by defining a proper time unit (slot) and representing the task durations using integer multiples of this unit.

Synchronization Constraint. Tasks can be processed in parallel on their corresponding machines; however, a job is considered completed only when all of its tasks finish. Hence the completion time of job j, denoted by C_j, satisfies

$$C_j = \max_{k \in \mathcal{K}_j} C_{kj}, \tag{1}$$

where C_{kj} is the completion time of its task (k, j).

Packing Constraint. The sum of resource requirements of tasks running in machine i should not exceed its capacity.

Let $\mathbb{1}(i \in \mathcal{M}_{kj})$ be 1 if $i \in \mathcal{M}_{kj}$, and 0 otherwise. Define the constant

$$T = \max_{i \in \mathcal{M}} \sum_{j \in \mathcal{J}} \sum_{k \in \mathcal{K}_j} p^i_{kj} \mathbb{1}(i \in \mathcal{M}_{kj}), \tag{2}$$

which is clearly an upper bound on the time required for processing all the jobs. We define 0-1 variables $X^i_{kj}(t)$, $i \in \mathcal{M}$, $j \in \mathcal{J}$, $k \in \mathcal{K}_j$, $t \leq T$, where $X^i_{kj}(t) = 1$ if task (k, j) is served at time slot t on machine i, and 0 otherwise. We also make the following definition.

Definition 2.1 (Height of Machine i at time t). The height of machine i at time t, denoted by $h_i(t)$, is the sum of resource requirements of the tasks running at time t in machine i, i.e.,

$$h_i(t) = \sum_{j \in \mathcal{J}, k \in \mathcal{K}_j} a_{kj} X^i_{kj}(t). \tag{3}$$

Given these definitions, a valid schedule $X^i_{kj}(t) \in \{0, 1\}$, $i \in \mathcal{M}$, $j \in \mathcal{J}$, $k \in \mathcal{K}_j$, $0 < t \leq T$, must satisfy the following three constraints:

(i) *Packing*: the sum of resource requirements of the tasks running in machine i at time t (i.e., tasks with $X^i_{kj}(t) = 1$) should not exceed machine i's capacity, i.e., $h_i(t) \leq m_i$, $\forall t \leq T$, $\forall i \in \mathcal{M}$.

(ii) *Placement*: each task at each time can get processed on at most a single machine selected from its feasible placement set, i.e., $\sum_{i \in \mathcal{M}_{kj}} X^i_{kj}(t) \leq 1$, and $X^i_{kj}(t) = 0$ if $i \notin \mathcal{M}_{kj}$.

(iii) *Processing*: each task must be processed completely. Noting that $X^i_{kj}(t)/p^i_{kj}$ is the fraction of task (k, j) completed on machine i in time slot t, we need $\sum_{i \in \mathcal{M}_{kj}} \sum_{t=1}^{T} X^i_{kj}(t)/p^i_{kj} = 1$.

Preemption and Migration. We consider three classes of scheduling policies. In a non-preemptive policy, a task cannot be preempted (and hence cannot be migrated among machines) once it starts processing on its corresponding machine until it is completed. In a preemptive policy, a task may be preempted and resumed several times in the schedule, and we can further consider two subcases depending on whether migration of a task among machines is allowed or not. Note that when migration is not allowed, the scheduler must assign each task (k, j) to one machine $i \in \mathcal{M}_{kj}$ on which the task is (preemptively or non-preemptively) processed until completion.

Main Objective. Given positive weights w_j, $j \in \mathcal{J}$, our goal is to find valid non-preemptive and preemptive (under with and without migrations) schedules of jobs (their tasks) in machines, so as to minimize the sum of weighted completion times of jobs, i.e.,

$$\text{minimize} \sum_{j \in \mathcal{J}} w_j C_j. \tag{4}$$

The weights can capture different priorities for jobs. Clearly the case of equal weights reduces the problem to minimization of the average completion time.

3 SCHEDULING WHEN MIGRATION IS ALLOWED

We first consider the case that migration of tasks among machines is allowed. In this case, we propose a preemptive algorithm, called SynchPack-1, with approximation ratio $(6+\epsilon)$ for any $\epsilon > 0$. We will use the construction ideas and analysis arguments for this algorithm to construct our preemptive and non-preemptive algorithms when migration is prohibited in Section 4.

In order to describe SynchPack-1, we first present a relaxed linear program. We will utilize the optimal solution to this LP to schedule tasks in a preemptive fashion.

3.1 Relaxed Linear Program (LP1)

Recall that without loss of generality, the processing times of tasks are assumed to be integers (multiples of a time unit) and therefore $C_j \geq p^i_{kj} \geq 1$ for all $j \in \mathcal{J}$, $k \in \mathcal{K}_j$, and $i \in \mathcal{M}_{kj}$. We use interval indexed variables using geometrically increasing intervals (see, e.g., [29, 30]) to formulate a linear program for our problem.

Let $\epsilon > 0$ be a constant. We choose L to be the smallest integer such that $(1+\epsilon)^L \geq T$ (recall T in (2)). Subsequently define

$$d_l = (1+\epsilon)^l, \text{ for } l = 0, 1, \cdots, L, \tag{5}$$

and define $d_{-1} = 0$. We partition the time horizon into time intervals $(d_{l-1}, d_l]$, $l = 0, ..., L$. Note that the length of the l-th interval, denoted by Δ_l, is

$$\Delta_0 = 1, \;\; \Delta_l = \epsilon(1+\epsilon)^{l-1} \quad \forall l \geq 1. \tag{6}$$

We define z^{il}_{kj} to be the fraction of task (k, j) (*fraction of its required processing time*) that is processed in interval l on machine $i \in \mathcal{M}_{kj}$. To measure completion time of job j, we define a variable x_{jl} for each interval l and job j such that, $\forall j \in \mathcal{J}$:

$$\sum_{l'=0}^{l} x_{jl'} \leq \sum_{l'=0}^{l} \sum_{i \in \mathcal{M}_{kj}} z^{il'}_{kj}, \; k \in \mathcal{K}_j, \; l = 0, \ldots, L \tag{7a}$$

$$\sum_{l=0}^{L} x_{jl} = 1, \;\; x_{jl} \in \{0, 1\}, \;\; l = 0, \cdots, L. \tag{7b}$$

Note that (7b) implies that only one of the variables $\{x_{jl}\}_{l=0}^{L}$ can be nonzero (equal to 1). (7a) implies that x_{jl} can be 1 only for one of the intervals $l \geq l^\star$ where $l^\star$ is the interval in which the last task of job j finishes its processing. Define,

$$C_j = \textstyle\sum_{l=0}^{L} d_{l-1} x_{jl} \;\; j \in \mathcal{J}. \tag{8}$$

If we can guarantee that $x_{jl^\star} = 1$ for $l^\star$ as defined above, then C_j will be equal to the starting point $d_{l^\star-1}$ of that interval, and the actual completion time of job j will be bounded above by $d_{l^\star} = (1+\epsilon)C_j$, thus implying that C_j is a reasonable approximation for the actual completion time of job j. This can be done by minimizing the objective function in the following linear program (LP1):

$$\min \sum_{j \in \mathcal{J}} w_j C_j \quad \textbf{(LP1)} \tag{9a}$$

$$\sum_{l=0}^{L} \sum_{i \in \mathcal{M}_{kj}} z^{il}_{kj} = 1, \;\; k \in \mathcal{K}_j, \; j \in \mathcal{J} \tag{9b}$$

$$\sum_{l'=0}^{l} \sum_{i \in \mathcal{M}_{kj}} z^{il'}_{kj} p^i_{kj} \leq d_l, \;\; k \in \mathcal{K}_j, \; j \in \mathcal{J}, \; l = 0, \ldots, L \tag{9c}$$

$$\sum_{l'=0}^{l} \sum_{(k,j): i \in \mathcal{M}_{kj}} z^{il'}_{kj} p^i_{kj} a_{kj} \leq m_i d_l, \;\; i \in \mathcal{M}, \; l = 0, \ldots, L \tag{9d}$$

$$z^{il}_{kj} \geq 0, \;\; k \in \mathcal{K}_j, \; j \in \mathcal{J}, \; i \in \mathcal{M}_{kj}, \; l = 0, \ldots, L \tag{9e}$$

$$\sum_{l'=0}^{l} x_{jl'} \leq \sum_{l'=0}^{l} \sum_{i \in \mathcal{M}_{kj}} z^{il'}_{kj}, \; k \in \mathcal{K}_j, \; j \in \mathcal{J}, \; l = 0, \ldots, L \tag{9f}$$

$$C_j = \sum_{l=0}^{L} d_{l-1} x_{jl}, \;\; j \in \mathcal{J} \tag{9g}$$

$$\sum_{l=0}^{L} x_{jl} = 1, \;\; x_{jl} \geq 0, \;\; l = 0, \ldots, L, \; j \in \mathcal{J} \tag{9h}$$

Constraint (9b) means that each task must be processed completely. (9c) is because during the first l intervals, a task cannot be processed for more than d_l, the end point of interval l, which itself is due to requirement (ii) of Section 2. (9d) bounds the total volume of the tasks processed by any machine i in the first l intervals by $d_l \times m_i$. (9e) indicates that z variables have to be nonnegative.

Constraints (9f), (9h), (9g) are the relaxed version of (7a), (7b), (8), respectively, where the integral constraint in (7b) has been relaxed to (9h). To give more insight, note that (9f) has the interpretation of keeping track of the fraction of the job processed by the end of each time interval, which is bounded from above by the fraction of any of its tasks processed by the end of that time interval. We should finish processing of all jobs as indicated by (9h). Also (9g) computes a relaxation of the job completion time C_j, as a convex combination of the intervals' left points, with coefficients x_{jl}. Note that (9f) along with (9h) implies the fact that each task is processed completely.

3.2 Scheduling Algorithm: SynchPack-1

In the following, a *task fraction* (k, j, i, l) of task (k, j) corresponding to interval l, is a task with size a_{kj} and duration $z^{il}_{kj} p^i_{kj}$ that needs to be processed on machine i.

The SynchPack-1 (*Synchronized Packing*-1) algorithm has three main steps:

Step 1: Solve (LP1). We first solve (LP1) and obtain the optimal solution of $\{z^{il}_{kj}\}$ which we denote by $\{\tilde{z}^{il}_{kj}\}$.

Step 2: Pack task fractions greedily and construct schedule $\mathcal{S}$. To schedule task fractions, we use a greedy list scheduling policy as follows:

Consider an ordered list of the task fractions such that task fractions corresponding to interval l appear before the task fractions

corresponding to interval l', if $l < l'$. Task fractions within each interval and corresponding to different machines are ordered arbitrarily. The algorithm scans the list starting from the first task fraction, and schedules task fraction (k, j, i, l) on machine i, if some fraction of task (k, j) is not already scheduled on some other machine at that time, and machine i has sufficient capacity, i.e., $h_i(t) + a_{kj} \leq m_i$ (recall $h_i(t)$ in Definition 2.1). It then moves to the next task fraction in the list, repeats the same procedure, and so on. Upon completion of a task fraction, it preempts the task fractions corresponding to higher indexed intervals on all the machines if there is some unscheduled task fraction of a lower-indexed interval in the list. It then removes the completed task fraction from the list, updates the remaining processing times of the task fractions in the list, and start scheduling the updated list. This greedy list scheduling algorithm schedules task fractions in a preemptive fashion.

We refer to the constructed schedule as $\mathcal{S}$.

Step 3: Apply Slow Motion and construct schedule $\bar{\mathcal{S}}$. Unfortunately, we cannot bound the value of objective function (9a) for schedule $\mathcal{S}$ since completion times of some jobs in $\mathcal{S}$ can be very long compared to the completion times returned by (LP1). Therefore, we construct a new feasible schedule $\bar{\mathcal{S}}$, by stretching $\mathcal{S}$, for which we can bound the value of its objective function. This method is referred to as *Slow-Motion* technique [35]. Let $\tilde{Z}^i_{kj} = \sum_{l=0}^{L} \tilde{z}^{il}_{kj}$ denote the total fraction of task (k, j) that is scheduled in machine i according to the optimal solution to (LP1). We refer to $\tilde{Z}^i_{kj}$ as the *total task fraction* of task (k, j) on machine i. The Slow Motion works by choosing a parameter $\lambda \in (0, 1]$ randomly drawn according to the probability density function $f(\lambda) = 2\lambda$. It then stretches schedule $\mathcal{S}$ by a factor $1/\lambda$. If a task is scheduled in $\mathcal{S}$ during an interval $[\tau_1, \tau_2)$, the same task is scheduled in $\bar{\mathcal{S}}$ during $[\tau_1/\lambda, \tau_2/\lambda)$ and *the machine is left idle if it has already processed its total task fraction $\tilde{Z}^i_{kj}$ completely*. We may also shift back future tasks' schedules as far as the machine capacity allows and placement constraint is respecred.

A pseudocode for SynchPack-1 can be found in Appendix G. The obtained algorithm is a randomized algorithm; however, we will show in Appendix C how we can de-randomize it to get a deterministic algorithm.

3.3 Performance Guarantee

We now analyze the performance of SynchPack-1. The result is stated by the following proposition.

Theorem 3.1. *For any $\epsilon > 0$, the sum of weighted completion times of jobs, for the problem of parallel-task job scheduling with packing and placement constraints, under* SynchPack-1*, is at most $(6 + \epsilon) \times OPT$.*

The rest of the section is devoted to the proof of Theorem 3.1. We use $\tilde{C}_j$ to denote the optimal solution to (LP1) for completion time of job $j \in \mathcal{J}$. The optimal objective value of (LP1) is a lower bound on the optimal value of our scheduling problem as stated in the following lemma whose proof is provided in Appendix B.1.

Lemma 3.2. $\sum_{j=1}^{N} w_j \tilde{C}_j \leq \sum_{j=1}^{N} w_j C^\star_j = OPT$.

Note that Constraint (9d) bounds the volume of all the task fractions corresponding to the first l intervals by $d_l \times m_i$. However, the (LP1)'s solution does not directly provide a feasible schedule as task fractions of the same task on different machines might overlap during the same interval and machines' capacity constraints might be also violated. Next, we show under the greedy list scheduling policy (Step 2 in SynchPack-1), the completion time of task fraction (k, j, i, l) is bounded from above by $3 \times d_l$, i.e., we need a factor 3 to guarantee a feasible schedule.

Lemma 3.3. *Let τ_l denote the time that all the task fractions (k, j, i, l'), for $l' \leq l$, are completed in schedule $\mathcal{S}$. Then, $\tau_l \leq 3d_l$.*

Proof. Consider the non-zero task fractions (k, j, i, l'), $i \in \mathcal{M}$, $l' \leq l$ (according to an optimal solution to (LP1)). Without loss of generality, we normalize the processing times of task fractions to be positive integers, by defining a proper time unit and representing the task durations using integer multiples of this unit. Let D_l and T_l be the value of d_l and τ_l using the new unit. Let $i^\star$ denote the machine that schedules the last task fraction among the non-zero task fractions of the first l intervals. Note that T_l is the time that this task fraction completes. If $T_l \leq D_l$, then $T_l \leq 3D_l$ and the lemma is proved. Hence consider the case that $T_l > D_l$.

Define $h_{il}(t)$ to be the height of machine i at time t in schedule $\mathcal{S}$ considering only the task fractions of the first l intervals. First we note that,

$$\sum_{l'=0}^{l} \sum_{(k,j): i \in \mathcal{M}_{kj}} z^{il'}_{kj} p^i_{kj} a_{kj} \overset{(a)}{=} \sum_{t=1}^{T_l} h_{il}(t) \overset{(b)}{\leq} m_i D_l, \quad \forall i \in \mathcal{M} \tag{10}$$

Using the definition of $h_{il}(t)$, the right-hand side of Equality (a) is the total volume of task fractions corresponding to the first l intervals that are processed during the interval $(0, T_l]$ on machine i, which is the left-hand side. Further, Inequality (b) is by Constraint (9d).

Let $S_{il}(\theta)$ denote the set of task fractions, running at time θ on machine i. Consider machine $i^\star$. We construct a bipartite graph $G = (U \cup V, E)$[1] as follows. For each time slot $\theta \in \{1, \ldots, T_l\}$, we consider a node z_θ, and define $V = \{z_\theta | 1 \leq \theta \leq T_l - D_l\}$, and $U = \{z_\theta | T_l - D_l + 1 \leq \theta \leq T_l\}$. For any $z_s \in U$ and $z_t \in V$, we add an edge (z_s, z_t) if $h_{i^\star l}(s) + h_{i^\star l}(t) \geq m_{i^\star}$. Note that $h_{i^\star l}(s) + h_{i^\star l}(t) < m_{i^\star}$ means, by definition, that there is no edge between z_s and z_t. Therefore, in this case,

$$\big(\cup_{i \in \mathcal{M}} S_{il}(s)\big) \setminus \big(\cup_{i \in \mathcal{M}} S_{il}(t)\big) = \varnothing. \tag{11}$$

This is because otherwise SynchPack-1 would have scheduled the task(s) in $S_{i^\star l}(s)$ at time t (note that $t < s$).

For any set of nodes $\tilde{U} \subseteq U$, we define set of its neighbor nodes as $N_{\tilde{U}} = \{z_t \in V | \exists\, z_s \in \tilde{U} : (z_s, z_t) \in E\}$. Note that, there are $T_l - D_l - |N_{\tilde{U}}|$ nodes in V which do not have any edge to some node in $\tilde{U}$. Let $|\cdot|$ denote set cardinality (size). We consider two cases:

Case (i): There exists a set $\tilde{U}$ for which $|N_{\tilde{U}}| < |\tilde{U}|$. Consider a node $z_s \in \tilde{U}$ and a task with duration p running at time slot s. Let p_U denote the amount of time that this task is running on time slots of set U. Note that $p_U \geq 1$. By Equation (11), a task that is

[1] $G = (U \cup V, E)$ is a bipartite graph iff for any edge $e = (u, v) \in E$, we have $u \in U$ and $v \in V$.

running at time s is also running at $T_l - D_l - |N_{\tilde{U}}|$ many other time slots whose corresponding nodes are in V.

$$p = T_l - D_l - |N_{\tilde{U}}| + p_U \leq D_l,$$

where the inequality is by Constraint (9c). Therefore

$$T_l \leq 2D_l + |N_{\tilde{U}}| - p_U < 2D_l + |\tilde{U}| \leq 3D_l.$$

Case (ii): For any $\tilde{U} \subseteq U$, $|\tilde{U}| \leq |N_{\tilde{U}}|$. Hence, $|V| \geq |U|$ which implies that $T_l \geq 2D_l$. Further, Hall's Theorem [20] states that a perfect matching of nodes in U to nodes in V always exists in G^2 in this case. The existence of such a matching then implies that any time slot $s \in (T_l - D_l, T_l]$ can be matched to a time slot $t_s \in (0, T_l - D_l]$ and $h_{i^\star l}(s) + h_{i^\star l}(t_s) \geq m_i$. This implies that

$$\sum_{s \in U} (h_{i^\star l}(s) + h_{i^\star l}(t_s)) \geq m_{i^\star} D_l \overset{(c)}{\geq} \sum_{t=1}^{T_l} h_{i^\star l}(t), \tag{12}$$

where Inequality (c) is by Equation (10). From this, one can conclude that no non-zero task fraction $(k, j, i^\star, l')$, $i^\star$, $l' \leq l$ is processed at time slots $V' = V \setminus \cup_{s \in U} \{t_s\}$. Hence, $V' = \varnothing$, since otherwise SynchPack-1 would have scheduled some of the tasks running at time slots of set U, at V'. We then can conclude that $T_l = 2D_l < 3D_l$. This completes the proof. □

Next, we make the following definition regarding schedule $\mathcal{S}$.

Definition 3.4. We define $C_j(\alpha)$, for $0 < \alpha \leq 1$, to be the time at which α-fraction of job j is completed in schedule $\mathcal{S}$ (i.e., at least α-fraction of each of its tasks has been completed.).

The following lemma shows the relationship between $C_j(\alpha)$ and $\tilde{C}_j$, the optimal solution to (LP1) for completion time of job j. The proof is provided in Appendix B.2.

Lemma 3.5. $\int_{\alpha=0}^{1} C_j(\alpha) d\alpha \leq 3(1+\epsilon)\tilde{C}_j$

Recall that schedule $\bar{\mathcal{S}}$ is formed by stretching schedule $\mathcal{S}$ by factor $1/\lambda$. Let $\bar{C}_j^\lambda$ denote the completion time of job j in $\bar{\mathcal{S}}$. Then we can show that the following lemma holds.

Lemma 3.6. $\mathbb{E}\left[\bar{C}_j^\lambda\right] \leq 6(1+\epsilon)\tilde{C}_j$.

Proof. The proof is based on Lemma 3.5 and taking expectation with respect to probability density function of λ. The details can be found in Appendix B.3. □

In constructing $\bar{\mathcal{S}}$, we may shift scheduling time of some of the tasks on each machine to the left and construct a better schedule. Nevertheless, we have the performance guarantee of Theorem 3.1 even without this shifting.

Proof of Theorem 3.1. Let C_j denote the completion time of job j under SynchPack-1. Then

$$\mathbb{E}\Big[\sum_{j \in \mathcal{J}} w_j C_j\Big] \leq \mathbb{E}\Big[\sum_{j \in \mathcal{J}} w_j \bar{C}_j^\lambda\Big] \overset{(a)}{\leq} 6(1+\epsilon) \sum_{j \in \mathcal{J}} w_j \tilde{C}_j \overset{(b)}{\leq} 6(1+\epsilon) \sum_{j \in \mathcal{J}} w_j C_j^\star,$$

where (a) is by Lemma 3.6, and (b) is by Lemma 3.2. In Appendix C, we discuss how to de-randomize the random choice of $\lambda \in (0, 1]$, which is used to construct schedule $\bar{\mathcal{S}}$ from schedule $\mathcal{S}$. So the proof is complete. □

[2] A perfect matching in G (with size $|U|$) is a subset of E such that every node in set U is matched to one and only one node in set V by an edge in the subset.

4 SCHEDULING WHEN MIGRATION IS NOT ALLOWED

The algorithm in Section 3 is preemptive, and tasks can be migrated across the machines in the same placement set. Implementing such an algorithm can be complex and costly in practice. In this section, we consider the case that migration of tasks among machines is not allowed. We propose a non-preemptive scheduling algorithm for this case. We also show that its solution provides a bounded solution for the case that preemption of tasks (in the same machine, without migration) is allowed.

Our algorithm is based on a relaxed LP which is very similar to (LP1) of Section 3, however a different constraint is used to ensure that each task is scheduled entirely by the end point of some time-interval of a machine. Next, we introduce this LP and describe how to generate a non-preemptive schedule based on its solution.

4.1 Relaxed Linear Program (LP2)

We partition the time horizon into intervals $(d_{l-1}, d_l]$ for $l = 0, ..., L$, as defined in (5) by replacing ϵ by 1. Define 0-1 variable z_{kj}^{il} to indicate whether task (k, j) is completed on machine i by the end-point of interval l, i.e., by d_l. Note that the interpretation of variables z_{kj}^{il} is slightly different from their counterparts in (LP1). By relaxing integrality of z variables, we formulate the following LP:

$$\min \sum_{j \in \mathcal{J}} w_j C_j \qquad \textbf{(LP2)} \tag{13a}$$

$$z_{kj}^{il} = 0 \ \text{ if } p_{kj}^i > d_l, \ \ j \in \mathcal{J}, \ k \in \mathcal{K}_j, \ i \in \mathcal{M}_{kj}, \ l = 0, \ldots, L \tag{13b}$$

$$\text{Constraints (9b)–(9h)} \tag{13c}$$

Note that Constraint (13b) allows z_{kj}^{il} to be positive only if the end point of l-th interval is at least as long as task (k, j)'s processing time on machine $i \in \mathcal{M}_{kj}$. We would like to emphasize that this is a valid constraint for both the preemptive and non-preemptive cases when migration is not allowed. We will see shortly how this constraint helps us construct our non-preemptive algorithm. We interpret *fractional* values of z_{kj}^{il} as the fraction of task (k, j) that is processed in interval l of machine i (as in Section 3).

4.2 Scheduling Algorithm: SynchPack-2

Our non-preemptive algorithm, which we refer to as SynchPack-2, has three main steps:

Step 1: Solve (LP2). We first solve the linear program (LP2) to obtain the optimal solution of $\{z_{kj}^{il}\}$ denoted by $\{\tilde{z}_{kj}^{il}\}$.

Step 2: Apply Slow-Motion. Before constructing the actual schedule of tasks, the algorithm applies the Slow-Motion technique (see Section 3.2). We pause here to clarify the connection between $\tilde{z}_{kj}^{il}$ and those obtained after applying Slow-Motion which we denote by $\bar{z}_{kj}^{il}$, below.

Recall that $\tilde{z}^{il}_{kj}$ is the fraction of task (k,j) that is scheduled in interval l of machine i in the optimal solution to (LP2), and Δ_l is the length of the l-th interval. Also, recall that $\tilde{Z}^i_{kj} = \sum_{l=0}^{L} \tilde{z}^{il}_{kj}$ is the total task fraction to be scheduled on machine i corresponding to task (k,j). Similarly, we define $\bar{\Delta}_l$ and $\bar{d}_l$ to be the length and the end point of the l-th interval after applying the Slow-Motion using a stretch parameter $\lambda \in (0,1]$, respectively. Therefore,

$$\bar{\Delta}_l = \frac{\Delta_l}{\lambda}, \quad \bar{d}_l = \frac{d_l}{\lambda}. \tag{14}$$

Further, we define $\bar{z}^{il}_{kj}$ to be the fraction of task (k,j) to be scheduled during the l-th interval on machine i after applying Slow-Motion. Then it holds that,

$$\bar{z}^{il}_{kj} = \begin{cases} \frac{\tilde{z}^{il}_{kj}}{\lambda}, & \text{if } \sum_{l\prime=0}^{l} \frac{\tilde{z}^{il\prime}_{kj}}{\lambda} < \tilde{Z}^i_{kj} \\ \max\left\{0, \left(\tilde{Z}^i_{kj} - \sum_{l\prime=0}^{l-1} \frac{\tilde{z}^{il\prime}_{kj}}{\lambda}\right)\right\}, & \text{otherwise.} \end{cases} \tag{15}$$

To see (15), note that in Slow-Motion, both variables and intervals are stretched by factor $1/\lambda$, and after stretching, the machine is left idle if it has already processed its total task fraction completely. Hence, as long as $\tilde{Z}^i_{kj}$ fraction of task (k,j) is not completely processed by the end of the l-th interval in the stretched solution, it is processed for $\tilde{z}^{il}_{kj} p^i_{kj}/\lambda$ amount of time in the l-th interval of length $\bar{\Delta}_l = \Delta_l/\lambda$. Hence $\bar{z}^{il}_{kj} = \tilde{z}^{il}_{kj}/\lambda$. Now suppose $l^\star$ is the first interval for which $\sum_{l\prime=0}^{l^\star} \tilde{z}^{il\prime}_{kj}/\lambda \geq \tilde{Z}^i_{kj}$. Then, the remaining processing time of task (k,j) to be scheduled in the $l^\star$-th interval of machine i in the stretched schedule is $p^i_{kj}(\tilde{Z}^i_{kj} - \sum_{l\prime=0}^{l^\star-1} \tilde{z}^{il\prime}_{kj}) = p^i_{kj}(\tilde{Z}^i_{kj} - \sum_{l\prime=0}^{l^\star-1} \tilde{z}^{il\prime}_{kj}/\lambda) > 0$. Therefore, the second part of (15) holds for $l^\star$, and for intervals $l > l^\star$, $\bar{z}^{il}_{kj}$ will be zero, since $\tilde{Z}^i_{kj} - \sum_{l\prime=0}^{l-1} \tilde{z}^{il\prime}_{kj}/\lambda \leq 0$. Observe that $\sum_{i\in\mathcal{M}_{kj}} \sum_{l=0}^{L} \bar{z}^{il}_{kj} = 1$.

Step 3: Construct a non-preemptive schedule. Note that according to variables $\bar{z}^{il}_{kj}$, a task possibly is set to get processed in different intervals and machines. The last step of SynchPack-2 is the procedure of constructing a non-preemptive schedule using these variables. This procedure involves 2 substeps: (1) *mapping of tasks to machine-intervals*, and (2) *non-preemptive scheduling of tasks mapped to each machine-interval using a greedy scheme*. We now describe each of these substeps in detail.

Substep 3.1: Mapping of tasks to machine-intervals. For each task (k,j), the algorithm uses a mapping procedure to find *a machine and an interval in which it can schedule the task entirely in a non-preemptive fashion*. The mapping procedure is based on constructing a weighted bipartite graph $\mathcal{G} = (U \cup V, E)$, followed by an *integral matching* of nodes in U to nodes in V on edges with non-zero weights, as described below:

(i) *Construction of Graph* $\mathcal{G} = (U \cup V, E)$*:* For each task (k,j), $j \in \mathcal{J}$, $k \in \mathcal{K}_j$, we consider a node in U. Therefore, there are $\sum_{j\in\mathcal{J}} |\mathcal{K}_j|$ nodes in U. Further, $V = \cup_{i\in\mathcal{M}} V_i$, where V_i is the set of nodes that we add for machine i. To construct graph $\mathcal{G}$, we start from the first machine, say machine i, and sort tasks in non-increasing order of their volume $v^i_{kj} = a_{kj} p^i_{kj}$ in machine i. Let N_i denote the number of tasks on machine i with nonzero volumes. Without loss of generality, suppose

$$v^i_{k_1 j_1} \geq v^i_{k_2 j_2} \geq \ldots v^i_{k_{N_i} j_{N_i}} > 0. \tag{16}$$

For each interval l, we consider $\lceil \bar{z}^{il} \rceil = \lceil \sum_{j\in J} \sum_{k\in\mathcal{K}_j} \bar{z}^{il}_{kj} \rceil$ (recall the definition of $\bar{z}^{il}_{kj}$ in (15)) consecutive nodes in V_i which we call *copies of interval* l.

Starting from the first task in the ordering (16), we draw edges from its corresponding node in U to the interval copies in V_i in the following manner. Assume we reach at task (k,j) in the process of adding edges. For each interval l, if $\bar{z}^{il}_{kj} > 0$, first set $R = \bar{z}^{il}_{kj}$. Consider the first copy of interval l for which the total weight of its current edges is strictly less than 1 and set W to be its total weight. We draw an edge from the node of task (k,j) in U to this copy node in V_i, and assign a weight equal to $\min\{R, 1-W\}$ to this edge. Then we update $R \leftarrow R - \min\{R, 1-W\}$, consider the next copy of interval l, and apply the same procedure, until $R = 0$ (or equivalently, the sum of edge weights from node (k,j) to copies of interval l becomes equal to $\bar{z}^{il}_{kj}$). We use w^{ilc}_{kj} to denote the weight of edge that connects task (k,j) to copy c of interval l of machine i, and if there is no such edge, $w^{ilc}_{kj} = 0$. We then move to the next machine and apply the similar procedure, and so on. See Figure 1 for an illustrative example.

Note that in $\mathcal{G}$, the weight of any node $u \in U$ (the sum of weights of its edges) is equal to 1 (since $\sum_{l=0}^{L} \bar{z}^{il}_{kj} = 1$, for any task (k,j)), while the weight of any node $v \in V$ is at most 1.

(ii) *Integral Matching:* Finally, we find an integral matching on the non-zero edges of $\mathcal{G}$, such that each non-zero task is matched to some interval copy. As we will show shortly in Section 4.3, we can always find an integral matching of size $\sum_{j\in\mathcal{J}} |\mathcal{K}_j|$, the total number of tasks, in $\mathcal{G}$, in polynomial time, in which each task is matched to a copy of some interval.

A pseudocode for the mapping procedure can be found in Appendix H.

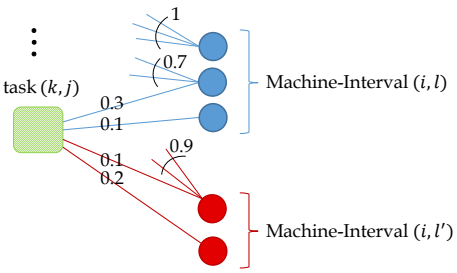


Figure 1: An illustrative example for construction of graph $\mathcal{G}$ in Substeb 3.1. Task (k,j) requires $\bar{z}^{il}_{kj} = 0.4$ and $\bar{z}^{il'}_{kj} = 0.3$. When we reach at task (k,j), the total weight of the first copy of interval l is 1 and that of its second copy is 0.7. Also, the total weight of the first copy of interval l' is 0.9. Hence, the procedure adds 2 edges to copies of interval l with weights 0.3 and 0.1, and 2 edges to copies of interval l' with weights 0.1 and 0.2.

Substep 3.2: Greedy packing of tasks in machine-intervals. We utilize a greedy packing to schedule all the tasks that are mapped

to a machine-interval *non-preemptively*. More precisely, on each machine, the greedy algorithm starts from the first interval and considers an arbitrary ordered list of its corresponding tasks. Starting from the first task, the algorithm schedules it, and moves to the second task. If the machine has sufficient capacity, it schedules the task, otherwise it checks the next task and so on. Once it is done with all the tasks of the first interval, it considers the second interval, applies the similar procedure, and so on. We may also shift back future tasks' schedules as far as the machine capacity allows.

Note that this greedy algorithm is simpler than the one described in Section 3, since it does not need to consider requirement (ii) of Section 2 as here each task only appears in one feasible machine.

As we prove in the next section, we can bound the total volume of tasks mapped to interval l on machine i in the mapping phase by $m_i\bar{\Lambda}_l$. Furthermore, by Constraint (13b) and the fact that the integral matching in Substep 3.1 was constructed on non-zero edges, the processing time of any task mapped to an interval is not greater than the interval's end point, which is twice the interval length. Hence, we can bound the completion time of each job and find the approximation ratio that our algorithm provides.

A pseudocode for the SynchPack-2 algorithm can be found in Appendix H.

4.3 Performance Guarantee

In this section, we analyze the performance of our non-preemptive algorithm SynchPack-2. The main result of this section is as follows:

Theorem 4.1. *The scheduling algorithm* SynchPack-2*, in Section 4.2, is a* 24*-approximation algorithm for the problem of parallel-task jobs scheduling with packing and placement constraints, when preemption and migration is not allowed.*

Since the constraints of (LP2) also hold for the preemptive case when migration is not allowed, the optimal solution of this case is also lower bounded by the optimal solution to the LP. Therefore, the algorithm' solution is also a bounded solution for the case that preemption is allowed (while still migration is not allowed).

Corollary 4.2. *The scheduling algorithm* SynchPack-2*, in Section 4.2, is a* 24*-approximation algorithm for the problem of parallel-task jobs scheduling with packing and placement constraints, when preemption is allowed and migration is not.*

The rest of this section is devoted to the proof of Theorem 4.1. With a minor abuse of notation, we use $\tilde{C}_{kj}$ and $\tilde{C}_j$ to denote the completion time of task (k, j) and job j, respectively, in the optimal solution to (LP2). Also, let $C^{\star}_{kj}$ and $C^{\star}_j$ denote the completion time of task (k, j) and job j, respectively, in the optimal non-preemptive schedule. We can bound the optimal value of (LP2) as stated below. The proof is provided in Appendix D.1.

Lemma 4.3. $\sum_{j=1}^{N} w_j\tilde{C}_j \le \sum_{j=1}^{N} w_jC^{\star}_j = OPT$.

Definition 4.4. Given $0 < \alpha \le 1$, define $\hat{C}_j(\alpha)$ to be the starting point of the earliest interval l for which $\alpha \le \tilde{x}_{jl}$ in solution to (LP2).

Note that $\hat{C}_j(\alpha)$ is slightly different from Definition 3.4, as we do not construct an actual schedule yet. We then have the following corollary which is a counterpart of Lemma 3.5. See Appendix D.2 for the proof.

Corollary 4.5. $\int_{\alpha=0}^{1} \hat{C}_j(\alpha)d\alpha = \tilde{C}_j$

Consider the mapping procedure where we construct bipartite graph $\mathcal{G}$ and match each task to a copy of some machine-interval. Below, we state a lemma which ensures that indeed we can find an integral (i.e. 0 or 1) matching in $\mathcal{G}$. The proof can be found in Appendix D.3.

Lemma 4.6. *Consider graph $\mathcal{G}$ constructed in the mapping procedure. There exists an integral matching on the nonzero edges of $\mathcal{G}$ in which each task is matched to some interval copy. Further, this matching can be found in polynomial time.*

Let V_{il} denote the total volume of the tasks mapped to all the copies of interval l of machine i. The following lemma bounds V_{il} whose proof is provided in Appendix D.4.

Lemma 4.7. *For any machine-interval (i, l), we have*

$$V_{il} \le \bar{d}_l m_i + \sum_{j\in\mathcal{J}} \sum_{k\in\mathcal{K}_j} v^i_{kj}\bar{z}^{il}_{kj}. \tag{17}$$

Note that the second term in the right side of (17) can be bounded by $\bar{d}_l m_i$ which results in the inequality $V_{il} \le 2\bar{d}_l m_i$. However, the provided bound is tighter and allows us to prove a better bound for the algorithm. We next show that, using the greedy packing algorithm, we can schedule all the tasks of an interval l in a bounded time.

In the case of packing single tasks in a single machine, the greedy algorithm is known to provide a 2-approximation solution for minimizing makespan [16]. The situation is slightly different in our setting as we require to bound the completion time of the last task as a function of the total volume of tasks, when the maximum duration of all tasks in each interval is bounded. We state the following lemma and its proof in Appendix D.5 for completeness.

Lemma 4.8. *Consider a machine with capacity* 1 *and a set of tasks $J = \{1, 2, \ldots, n\}$. Suppose each task j has size $a_j \le 1$, processing time $p_j \le 1$, and $\sum_{j\in J} a_jp_j \le v$. Then, we can schedule all the tasks within the interval $(0, 2\max\{1, v\}]$ using the greedy algorithm.*

Now consider a machine-interval (i, l). Note that Lemma 4.7 bounds the total volume of tasks while Constraint (13b) ensures that duration of each task is less than d_l. Thus, by applying Lemma 4.8 on the normalized instance, in which size and length of tasks are normalized by m_i and d_l, respectively, we guarantee that we can schedule all the task within a time interval of length $2d_l + 2\sum_{j\in\mathcal{J}}\sum_{k\in\mathcal{K}_j} v^i_{kj}\bar{z}^{il}_{kj}/m_i$. Moreover, the factor 2 is tight as stated in the following lemma. The proof can be found in Appendix D.6.

Lemma 4.9. *We need an interval of length at least* $2\max(1, v)$ *to be able to schedule any list of tasks as in Lemma 4.8 using any algorithm.*

Hence, Lemmas 4.8 and 4.9 imply that applying the greedy algorithm to schedule the tasks of each machine-interval, provides a tight bound with respect to the total volume of tasks in that machine-interval. Let C_{kj} denote the completion time of task (k, j) under SynchPack-2. Then we have the following lemma.

Lemma 4.10. *Suppose that task (k, j) is mapped to the l-th interval of machine i at the end of Substep 3.1. Then, $C_{kj} \le 6\bar{d}_l$.*

Proof. Let T_{il} denote the completion time of the last task of machine-interval (i,l), and $\tau_{il'}$ be the length of the time interval that SynchPack-2 uses to schedule tasks of machine-interval (i,l). Then,

$$C_{kj} \le T_{il} = \sum_{l'=0}^{l} \tau_{il'} \overset{(a)}{\le} 2 \times \sum_{l'=0}^{l} \Big(\bar{d}_{l'} + \sum_{j' \in \mathcal{J}} \sum_{k' \in \mathcal{K}_{j'}} v^i_{k'j'} \bar{z}^{il'}_{k'j'}/m_i\Big)$$
$$\overset{(b)}{\le} 4\bar{d}_l + 2 \sum_{l'=0}^{l} \sum_{j' \in \mathcal{J}} \sum_{k' \in \mathcal{K}_{j'}} v^i_{k'j'} \bar{z}^{il'}_{k'j'}/m_i \overset{(c)}{\le} 6\bar{d}_l. \tag{18}$$

Inequality (a) is due to Lemma 4.7 and Lemma 4.8, while Inequality (b) is because $d_{l'-1} = d_{l'}/2$. Further, Inequality (c) is by Constraint (9d). □

Proof of Theorem 4.1. Let l denote the end point of the interval in which task (k,j) has the last non-zero fraction according to $\bar{z}^{il}_{kj}$. Then,

$$\bar{d}_l = 2^l/\lambda \overset{(\star)}{\le} 2\hat{C}_j(\lambda)/\lambda. \tag{19}$$

First note that ϵ is replaced by 1 in Equation (5). Further, Inequality $(\star)$ follows from the definition of $\hat{C}_j(\lambda)$ (Definition 4.4), and the fact that d_l's are multiplied by $1/\lambda$. Therefore, $\hat{C}_j(\lambda)/\lambda$ is the start point of the interval in which job j is completed, and, accordingly, $2\hat{C}_j(\lambda)/\lambda$ is the end point of that interval. Thus, $2^l/\lambda$, the end point of the interval in which task (k,j) is completed, has to be at most $2\hat{C}_j(\lambda)/\lambda$, the end point of the interval in which job j is completed.

Let C_{kj} and C_j be the completion time of task (k,j) and job j under SynchPack-2. Recall that in the mapping procedure, we only map a task to some interval l' in which part of the task is assigned to that interval after Slow-Motion applied (in other words, $\bar{z}^{il'}_{kj} > 0$). Thus, task (k,j) that has its last non-zero fraction in interval l (by our assumption) is mapped to some interval $l' \le l$, because $\bar{z}^{il''}_{kj} = 0$ for intervals $l'' > l$. Suppose task (k_j, j) is the last task of job j and finishes in interval l_j in our non-preemptive schedule. Then, by Lemma 4.10 and Equation (14), we have $C_j = C_{i_j j} \le 6\bar{d}_l = \frac{6}{\lambda} 2^{l_j}$. Recall that $\tilde{C}_j$ denotes the completion time of job j in an optimal solution of (LP2). Hence,

$$\mathbb{E}\Big[\sum_{j \in \mathcal{J}} w_j C_j\Big] \le \mathbb{E}\Big[\sum_{j \in \mathcal{J}} w_j \frac{6}{\lambda} 2^{l_j}\Big] \overset{(a)}{\le} 12 \times \mathbb{E}\Big[\sum_{j \in \mathcal{J}} w_j \hat{C}_j(\lambda)/\lambda\Big]$$
$$\overset{(b)}{=} 12 \times \sum_{j \in \mathcal{J}} w_j \int_{\lambda=0}^{1} \frac{\hat{C}_j(\lambda)}{\lambda} 2\lambda d\lambda \overset{(c)}{\le} 24 \times \sum_{j \in \mathcal{J}} w_j \tilde{C}_j,$$

where in the above, (a) is by the second part of (19) for $l = l_j$, (b) is by definition of expectation with respect to λ, with pdf $f(\lambda) = 2\lambda$, and (c) is by Corollary 4.5. Using the above inequality and Lemma 4.3,

$$\mathbb{E}\Big[\sum_{j \in \mathcal{J}} w_j C_j\Big] \le 24 \times \sum_{j \in \mathcal{J}} w_j C^\star_j = 24 \times \mathrm{OPT}. \tag{20}$$

By applying de-randomization procedure (see Appendix C), we can find $\lambda = \lambda^\star$ in polynomial time for which the total weighted completion time is less that its expected value in (20). This completes the proof of Theorem 4.1. □

5 SPECIAL CASE: PREEMPTION AND SINGLE-MACHINE PLACEMENT SET

In previous sections, we studied the parallel-task job scheduling problem for both cases when migration of tasks (among machines in its placement set) is allowed or not, and provided $(6+\epsilon)$ and 24 approximation algorithms, respectively. In this section, we consider a special case when only one machine is in the placement set of each task (e.g., it is the only machine that has the required data for processing the task), and preemption is allowed.

Corollary 5.1. *Consider the parallel-task job scheduling problem when there is a specific machine to process each task and preemption is allowed. For any $\epsilon > 0$, the sum of the weighted completion times of jobs under* SynchPack-1*, in Section 3.2, is at most $(4+\epsilon) \times OPT$.*

Proof. The proof is straight forward and similar to proof of Theorem 3.1. Specifically, the factor 3 needed to bound the solution of the greedy policy is reduced to 2 due to the fact that placement constraint is not needed to be enforced here, since there is only one machine for each task. □

We can show that there is a slightly better approximation algorithm to solve the problem in this special case, that has an approximation ratio 4. The algorithm uses a relaxed LP, based on linear ordering variables (e.g., [15, 27, 38]) to find an efficient ordering of jobs. Then it applies a simple list scheduling to pack their tasks in machines subject to capacity constraints. The details are as follows.

5.1 Relaxed Linear Program (LP3)

Note that each task has to be processed in a specific machine. Each job consists of up to M (number of machines) different tasks. We use $\mathcal{M}_j$ to denote the set of machines that have tasks for job j. Task i of job j, denoted as task (i,j), requires a specific amount a_{ij} of machine iâĂŹs resource ($a_{ij} \le m_i$) for a specific time duration $p_{ij} > 0$. We also define its volume as $v_{ij} = a_{ij}p_{ij}$. The results also hold in the case that a job has multiple tasks on the same machine.

For each pair of jobs, we define a binary variable which indicates which job is finished before the other one. Specifically, we define $\delta_{jj'} \in \{0,1\}$ such that $\delta_{jj'} = 1$ if job j is completed before job j', and $\delta_{jj'} = 0$ otherwise. Note that by the synchronization constraint (1), the completion of a job is determined by its last task. If both jobs finish at the same time, we set either one of $\delta_{jj'}$ or $\delta_{j'j}$ to 1 and the other one to 0, arbitrarily. By relaxing the integral constraint on binary variables, we formulate the following LP:

$$\min \sum_{j \in \mathcal{J}} w_j C_j \qquad \textbf{(LP3)} \tag{21a}$$
$$m_i C_j \ge v_{ij} + \sum_{j' \in \mathcal{J}, j' \ne j} v_{ij'} \delta_{j'j}, \;\; j \in \mathcal{J}, i \in \mathcal{M}_j \tag{21b}$$
$$C_j \ge p_{ij}, \;\; j \in \mathcal{J}, i \in \mathcal{M}_j \tag{21c}$$
$$\delta_{jj'} + \delta_{j'j} = 1, \;\; j \ne j', \; j, j' \in \mathcal{J} \tag{21d}$$
$$\delta_{jj'} \ge 0, \;\; j, j' \in \mathcal{J} \tag{21e}$$

Recall the definition of job completion time C_j and task completion time C_{ij} in Section 2. In (LP3), (21b) follows from the definition of $\delta_{jj'}$, and the fact that the tasks which need to be served on machine i are processed by a single machine of capacity m_i. It states that

the total volume of tasks that can be processed during the time period $(0, C_j]$ by machine i is *at most* $m_i C_j$. This total volume is given by the right-hand-side of (21b) which basically sums the volumes of the tasks on machine i that finish before job j finishes its corresponding tasks at time C_j, plus the volume of task (i, j) itself. Constraint (21c) is due to the fact that $C_j \geq C_{ij}$ and each task cannot be completed before its processing time p_{ij}. (21d) indicates that for each two jobs, one precedes the other. Further, we relax the binary ordering variables to be fractional in (21e).

Note that the optimal solution to (LP3) might be an infeasible schedule as (LP3) replaces the tasks by sizes of their volumes and it might be impossible to pack the tasks in a way that matches the obtained completion times from (LP3).

Remark 1. (LP3) can be easily modified to allow each job to have multiple tasks on the same machine. We omit the details to focus on the main ideas.

5.2 Scheduling Algorithm: SynchPack-3

The SynchPack-3 algorithm has two steps:

Step 1: Solve (LP3) to find an ordering of jobs. Let $\tilde{C}_j$ denote the optimal solution to (LP3) for completion time of job $j \in \mathcal{J}$. We order jobs based on their $\tilde{C}_j$ values in a nondecreasing order. Without loss of generality, we re-index the jobs such that

$$\tilde{C}_1 \leq \tilde{C}_2 \leq \ldots \leq \tilde{C}_N. \tag{22}$$

Ties are broken arbitrarily.

Step 2: List scheduling based on the obtained ordering. For each machine i, the algorithm maintains a list of tasks such that for every two tasks (i, j) and (i, j') with $j < j'$ (according to ordering (22)), task (i, j) appears before task (i, j') in the list. On machine i, the algorithm scans the list starting from the first task. It schedules a task (i, j) from the list if the machine has sufficient remaining resource to accommodate it. Upon completion of a task, the algorithm preempts the schedule, removes the completed task from the list and updates the remaining processing time of the tasks in the list, and starts scheduling the tasks in the updated list. Observe that this list scheduling is slightly different from the greedy scheme used in SynchPack-1.

A pseudocode for the algorithm can be found in Appendix F.

5.3 Performance Guarantee

The main result regarding the performance of SynchPack-3 is summarized in the following theorem. The proof of the theorem, and any supporting lemmas, is presented in Appendix E.

Theorem 5.2. *The scheduling algorithm* SynchPack-3 *(Section 5.2) is a* 4*-approximation algorithm for the problem of parallel-task jobs scheduling with packing and single-machine placement constraints.*

6 COMPLEXITY OF ALGORITHMS

The complexity of our algorithms is mainly dominated by solving their corresponding LPs, which can be solved in polynomial time using efficient linear programming solvers. The rest of the operations have low complexity and can be parallelized on the machines. We have provided a detailed discussion of the complexity in Appendix A.

7 EVALUATION RESULTS

In this section, we evaluate the performance of our algorithms using a real traffic trace, and compare to prior algorithms.

Data Set. The data set is from a large Google cluster [41]. The original trace is over a month long period. To keep things simpler, we extract multi-task jobs of production scheduling class that were completed without any interruptions.In our experiments, we filter jobs and consider those with at most 200 tasks, which constitute about 99% of all the jobs in the production class. Also, in order to have reasonable traffic density on each machine (since otherwise the problem is trivial), we consider a cluster with 200 machines and randomly map machines of the original set to machines of this set. The final data set used for our simulations contains 7521 jobs with an average of 10 tasks per job. We also extracted memory requirement of each task and its corresponding processing time from the data set. In the data set, each job has a priority that represents its sensitivity to latency. There are 9 different values of job priorities.

In the original data set, each task is assigned to *one* machine, and the scheduler needs to decide when to start its scheduling. This is similar to our model for our preemptive algorithm SynchPack-3 in Section 5. To incorporate placement constraints, we modify the data set as follows. For each task, we randomly choose 3 machines and assume that processing time of the task on these machines is equal to the processing time given in the data set. We allow the task to be scheduled on other machines; however, its processing time will be penalized by a factor $\alpha > 1$. This is consistent with the data locality models in previous work (e.g. [18, 40]).

We evaluate the performance of algorithms in both offline and online settings. For the offline setting, we consider the first 1000 jobs in the data set and assume all of these jobs are in the system at time 0. For the online setting, all the 7521 jobs arrive according to the arrival times information in the data set. Further, we consider 3 different cases for weight assignments: 1) All jobs have equal weights, 2) Jobs are assigned random weights between 0 and 1, and 3) Jobs' weights are determined based on the job priority and class information in the data set.

Algorithms. We consider three prior algorithms, PSRS [37], Tetris [18], and JSQ-MW [40] to compare with our algorithms SynchPack-2 and SynchPack-3. We briefly overview the algorithms below.

1. PSRS [37]: *Preemptive Smith Ratio Scheduling* is a preemptive algorithm for the parallel task scheduling problem (see Section 1.1) on a single machine. Modified Smith ratio of task (i, j) is defined as $\frac{w_j}{a_{ij} p_{ij}} = \frac{w_j}{v_{ij}}$. Moreover, a constant $\mathcal{V} = 0.836$ is used in the algorithm. It also defines $T(a, t)$ to be the first time after t at which at least a amount of the machine's capacity is available, given the schedule at time t. On machine i, the algorithm first orders tasks based on the modified Smith ratio (largest ratio first). It then removes the first task (i, j) in the list and as long as the task needs at most 50% of the machine capacity m_i, it schedules the task in a non-preemptive fashion at the first time that available capacity of the machine is equal to or greater than the task's size, namely at $T(a_{ij}, t)$ where t is the current time and a_{ij} is the size of task (i, j). However, if task (i, j) requires more than half of the machine's capacity, the algorithm determines the difference $T(a_{ij}, t) - T(m_i/2, t)$. If this time difference is less than the ratio $p_{ij}/\mathcal{V}$, it schedules task (i, j)

in the same way as those tasks with smaller size; that is, (i, j) starts at $T(a_{ij}, t)$ and runs to completion. Otherwise at time $T(m_i/2, t) + p_{ij}/\mathcal{V}$, it preempts all the tasks that do not finish before that time, and starts task (i, j). After task (i, j) is completed, those preempted tasks are resumed. For the online setting, upon arrival of each task, the algorithm preempts the schedule, updates the list, and schedule the tasks in a similar fashion.

2. Tetris [18]: Tetris is a heuristic that schedules tasks on each machine according to an ordering based on their scores (Section 1.1). Tetris was originally designed for the case that all jobs have identical weights; therefore, we generalize it by incorporating weights in tasks' scores. For each task (i, j) at time t, its score is defined as $s_{ij} = w_j(a_{ij} + \frac{\epsilon}{\sum_i a_{ij} p_{ij}^t})$, where $\epsilon = \frac{\sum_i \sum_j w_j a_{ij}}{\sum_j w_j (\sum_i a_{ij} p_{ij}^t)^{-1}}$, and p_{ij}^t is the task's remaining processing time at time t. Note that the first term in the score depends on the task' size (it favors a larger task if it fits in the machine's remaining capacity), while the second term prefers a task whose job's remaining volume (based on the sum of its remaining tasks) is smaller. On each machine, Tetris orders tasks based on their scores and greedily schedules tasks according to the list as far as the machine capacity allows. We consider two versions of Tetris, preemptive (Tetris-p) and non-preemptive (Tetris-np). In Tetris-p, upon completion of a task (or arrival of a job, in the online setting), it preempts the schedule, update the list, calculate scores based on updated values, and schedule the tasks in a similar fashion. In Tetris-np, the algorithm does not preempt the tasks that are running; however, calculates scores for the remaining tasks based on updated values.

To take the placement constraint into account, Tetris imposes a remote penalty to the computed score to penalize use of remote resources. This remote penalty is suggested to be $\approx$ 10% in [18]. In simulations, we also simulated Tetris by penalizing scores by the factor α, and found out that the performance is slightly better. Hence, we only report performance of Tetris with this remote penalty.

3. JSQ-MW [40]: Join-the-Shortest-Queue routing with Max Weight scheduling (JSQ-MW) is a non-preemptive algorithm in presence of data locality (Section 1.1). It assigns an arriving task to the shortest queue among those corresponding to the 3 local servers with its input data and the remote queue. When a server is available, it either process a task from its local queue or from the remote queue, where the decision is made based on a MaxWeight. We further combine JSQ-MW with the greedy packing scheme so it can pack and schedule tasks non-preemptively in each server.

4. SynchPack-2 and SynchPack-3: These are our non-preemptive and preemptive algorithms as described in Section 4 and Section 5. The complexity of our algorithms is mainly dominated by solving their corresponding LPs. While (LP3) has reasonable size and can be solved quickly (see Section A for the details), (LP2) requires more memory for large instances. In this case, to expedite computation, besides the 3 randomly chosen local machines that can schedule a task, we consider 10 other machines (5% of the machines, instead of all the machines) that can process the task in an α times larger processing time. We choose these 10 machines randomly as well. Note that this may degrade the performance of our algorithm, nevertheless, as will see, they still significantly outperform the past algorithms.

A natural extension of our algorithms to online setting is as follows. We choose a parameter τ that is tunable. We divide time into time intervals of length τ. For the preemptive case, at the beginning of each interval, we preempt the schedule, update the processing times, and run the offline algorithm on a set of jobs, consisting of jobs that are not scheduled yet completely and those that arrived in the previous interval. In the non-preemptive case, tasks on the boundary of intervals are processed non-preemptively, i.e., we let the running tasks (according to the previously computed schedule) finish, then apply the non-preemptive offline algorithm on the updated list of jobs as in the preemptive online case, and proceed with the new schedule. Note that a larger value of τ reduces the complexity of the online algorithm; but it also decreases the overall performance. We use an adaptive choice of τ to improve the performance of our online algorithm, starting from smaller value of τ. In our simulations, we choose the length of the i-th interval, τ_i, as $\tau_i = \tau_0/(1 + \gamma \times \exp(-\beta i))$, $i = 1, 2, \cdots$, for some constants γ and β. We choose $\tau_0 = 3 \times 10^2$ seconds, which is 5 times greater than the average inter-arrival time of jobs, and $\gamma = 50$ and $\beta = 3$.

7.1 Results in Offline Setting

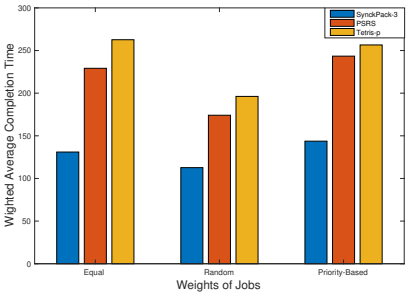
We use SynchPack-3, Tetris-p, and PSRS to schedule tasks of the original data set preemptively, and use SynchPack-2, Tetris-np, and JSQ-MW to schedule tasks of the modified data set (with placement constraints) non-preemptively. We then compare the weighted average completion time of jobs, $\sum_j w_j C_j / \sum_j w_j$, under these algorithms for the three weight cases, i.e. equal, random, and priority-based weights. Note that weighted average completion time is equivalent to the total weighted completion time (up to the normalization $\sum_j w_j$).

We first report the ratio between the total weighted completion time obtained from SynchPack-2 (for $\alpha = 2$) and SynchPack-3 and their corresponding optimal value of their relaxed LPs (13) and (21) (which is a lower bound on the optimal total weighted competition time) to verify Theorem 4.1 and 5.2. Table 1 shows this performance ratio for the 3 cases of job weights. All ratios are within our theoretical approximation ratio of 24 and 4. In fact, the approximation ratios are much smaller.

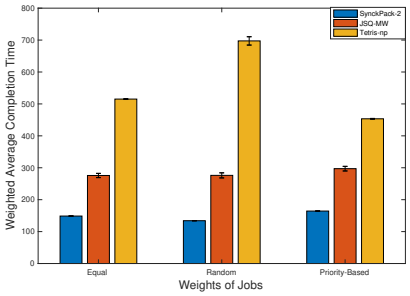
Table 1: Performance ratio between SynchPack-3 and (LP3), and between SynchPack-2 and (LP2)

Jobs' Weights	Equal	Random	Priority-Based
Ratio for SynchPack-2	2.87	2.90	2.98
Ratio for SynchPack-3	1.34	1.35	1.31

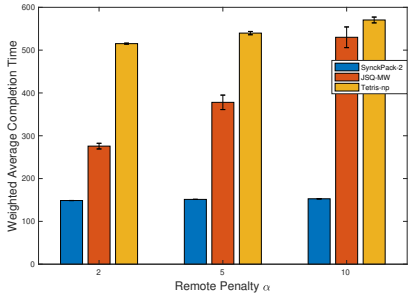
Figure 2a shows the performance of SynchPack-3, Tetris-p, and PSRS in the offline setting. As we see, SynchPack-3 outperforms the other two algorithms in all the cases and performance gain varies from 33% to 132%. Further, Figure 2b depicts performance of SynchPack-2, Tetris-np, and JSQ-MW for different weights, when $\alpha = 2$. The performance gain of SynchPack-2 varies from 81% to 420%. Figure 2c shows the effect of remote penalty α in the performance of SynchPack-2, Tetris-np, and JSQ-MW. As we see, SynchPack-2 outperforms the other algorithms by 85% to 273%



(a) Performance of SynchPack-3, Tetris-p, and PSRS for different weights.

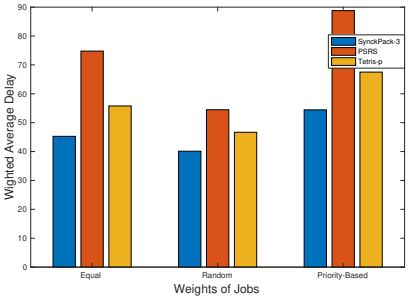


(b) Performance of SynchPack-2, Tetris-np, and JSQ-MW for different weights and remote penalty $\alpha = 2$.

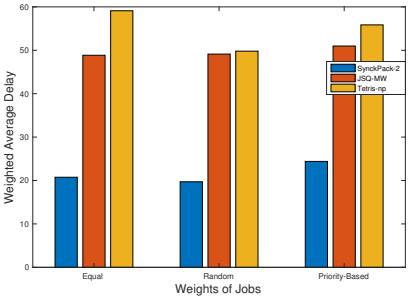


(c) Performance of SynchPack-2, Tetris-np, and JSQ-MW for different remote penalties and equal weights.

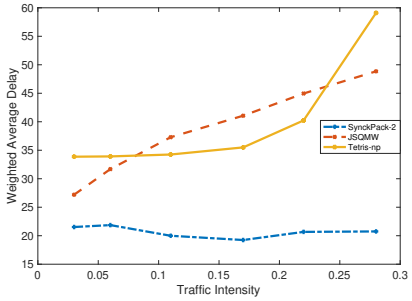
Figure 2: Performance of algorithms in the offline setting.



(a) Performance of SynchPack-3, Tetris-p, and PSRS for different weights.



(b) Performance of SynchPack-2, Tetris-np, and JSQ-MW for different weights.



(c) Performance of SynchPack-2, Tetris-np, and JSQ-MW for different traffic intensity.

Figure 3: Performance of algorithms in the online setting.

7.2 Results in Online Setting

In the online setting, jobs arrive dynamically over time, according to the arrival time information in the data set, and we are interested in the weighted average *delay* of jobs. The delay of a job is measured from the time that it arrives to the system until its completion.

Figure 3a shows the performance results, in terms of the weighted average *delay* of jobs, under SynchPack-3, Tetris-p, and PSRS. Performances of Tetris-p is worse than our algorithm by 11% to 27%, while PSRS presents the poorest performance and has 36% to 65% larger weighted average delay compared to SynchPack-3. Moreover, performance of SynchPack-2, Tetris-np, and JSQ-MW for different weights is depicted in Figure 3b. As we see, SynchPack-2 outperforms the other two algorithms in all the cases and performance gain varies from 109% to 189%.

Further, by multiplying arrival times by constant values we can change the traffic intensity and study its effect on algorithms' performance. Figure 3c shows the results for equal job weights. As we can see, SynchPack-2 outperforms the other algorithms and the performance gain increases as traffic intensity grows.

8 CONCLUSIONS

We studied the problem of scheduling jobs, each job with multiple resource constrained tasks, in a cluster of machines. Our motivation for this model came from modern parallel computing applications where tasks can be packed in machines, the input data required to process each task is stored in a subset of machines, and the tasks' collective completion time determines the job's completion. We proposed the first constant-approximation algorithms for minimizing the total weighted completion time of such jobs, namely, a $(6+\epsilon)$-approximation preemptive algorithm when task migration is allowed, and a 24-approximation non-preemptive (without migrations) algorithm. Further, for the special case when there is a specific machine for each task (single-machine data locality), we improved the approximation ratio for the preemptive case to 4. The model and analysis in our setting of tasks with packing, synchronization, and placement constraints is new. Note that the approximation results are upper bounds on the algorithms' performance. Our simulation results for our algorithms showed that the approximation ratios are in fact very close to 1 in practice.

As we showed, applying our simple greedy packing, to schedule tasks mapped to each interval in SynchPack-2, provides a tight bound on the total volume of tasks and its relation to the associated linear program. Therefore, we cannot improve the final result by replacing this step with more intelligent bin packing algorithms like BestFit [8]. Although, in practice, applying such bin packing schemes can give a better performance. Improving the performance bound of 24 requires a more careful and possibly different analysis. We leave further improvement of the result as a future work. Extension of our model to capture multi-dimensional task resource requirements ,and analysis of the online algorithms for our problem, are also interesting and challenging topics for future work.

REFERENCES

[1] Reza Ahmadi, Uttarayan Bagchi, and Thomas A Roemer. 2005. Coordinated scheduling of customer orders for quick response. *Naval Research Logistics (NRL)* 52, 6 (2005), 493–512.

[2] Ganesh Ananthanarayanan, Ali Ghodsi, Scott Shenker, and Ion Stoica. 2011. Disk-locality in datacenter computing considered irrelevant.. In *HotOS*, Vol. 13. 12–12.

[3] Ganesh Ananthanarayanan, Srikanth Kandula, Albert G Greenberg, Ion Stoica, Yi Lu, Bikas Saha, and Edward Harris. 2010. Reining in the Outliers in Map-Reduce Clusters using Mantri.. In *Osdi*, Vol. 10. 24.

[4] Nikhil Bansal and Subhash Khot. 2010. Inapproximability of hypergraph vertex cover and applications to scheduling problems. In *International Colloquium on Automata, Languages, and Programming*. Springer, 250–261.

[5] Jacek Blazewicz, Jan Karel Lenstra, and AHG Rinnooy Kan. 1983. Scheduling subject to resource constraints: classification and complexity. *Discrete applied mathematics* 5, 1 (1983), 11–24.

[6] Thomas Cheatham, Amr Fahmy, Dan Stefanescu, and Leslie Valiant. 1996. Bulk synchronous parallel computing–a paradigm for transportable software. In *Tools and Environments for Parallel and Distributed Systems*. Springer, 61–76.

[7] Zhi-Long Chen and Nicholas G Hall. 2007. Supply chain scheduling: Conflict and cooperation in assembly systems. *Operations Research* 55, 6 (2007), 1072–1089.

[8] Edward G Coffman, Jr, Michael R Garey, David S Johnson, and Robert Endre Tarjan. 1980. Performance bounds for level-oriented two-dimensional packing algorithms. *SIAM J. Comput.* 9, 4 (1980), 808–826.

[9] Jeffrey Dean and Sanjay Ghemawat. 2008. MapReduce: simplified data processing on large clusters. *Commun. ACM* 51, 1 (2008), 107–113.

[10] Apache Software Foundation. 2018. Apache Hadoop. http://hadoop.apache.org.

[11] Apache Software Foundation. 2018. Apache Hadoop Yarn. http://hadoop.apache.org/docs/current/hadoop-yarn/hadoop-yarn-site/YARN.html.

[12] Apache Software Foundation. 2018. Apache Spark. https://spark.apache.org/docs/latest/index.html.

[13] Apache Software Foundation. 2018. Hadoop Capacity Scheduler. https://hadoop.apache.org/docs/current/hadoop-yarn/hadoop-yarn-site/CapacityScheduler.html.

[14] Apache Software Foundation. 2018. Hadoop MapReduce-FairScheduler. http://hadoop.apache.org/docs/r2.4.1/hadoop-yarn/hadoop-yarn-site/FairScheduler.html.

[15] Rajiv Gandhi, Magnús M Halldórsson, Guy Kortsarz, and Hadas Shachnai. 2008. Improved bounds for scheduling conflicting jobs with minsum criteria. *ACM Transactions on Algorithms (TALG)* 4, 1 (2008), 11.

[16] Michael R Garey and Ronald L. Graham. 1975. Bounds for multiprocessor scheduling with resource constraints. *SIAM J. Comput.* 4, 2 (1975), 187–200.

[17] Naveen Garg, Amit Kumar, and Vinayaka Pandit. 2007. Order scheduling models: hardness and algorithms. In *International Conference on Foundations of Software Technology and Theoretical Computer Science*. Springer, 96–107.

[18] Robert Grandl, Ganesh Ananthanarayanan, Srikanth Kandula, Sriram Rao, and Aditya Akella. 2015. Multi-resource packing for cluster schedulers. *ACM SIGCOMM Computer Communication Review* 44, 4 (2015), 455–466.

[19] Robert Grandl, Srikanth Kandula, Sriram Rao, Aditya Akella, and Janardhan Kulkarni. 2016. {GRAPHENE}: Packing and dependency-aware scheduling for data-parallel clusters. In *12th {USENIX} Symposium on Operating Systems Design and Implementation ({OSDI} 16)*. 81–97.

[20] Philip Hall. 1935. On representatives of subsets. *Journal of the London Mathematical Society* 1, 1 (1935), 26–30.

[21] Jiahui Jin, Junzhou Luo, Aibo Song, Fang Dong, and Runqun Xiong. 2011. Bar: An efficient data locality driven task scheduling algorithm for cloud computing. In *2011 11th IEEE/ACM International Symposium on Cluster, Cloud and Grid Computing*. IEEE, 295–304.

[22] Karthik Kambatla, Naresh Rapolu, Suresh Jagannathan, and Ananth Grama. 2010. Asynchronous algorithms in MapReduce. In *2010 IEEE International Conference on Cluster Computing (CLUSTER)*. 245–254.

[23] Jan Karel Lenstra, David B Shmoys, and Eva Tardos. 1990. Approximation algorithms for scheduling unrelated parallel machines. *Mathematical programming* 46, 1-3 (1990), 259–271.

[24] Joseph Y-T Leung, Haibing Li, and Michael Pinedo. 2007. Scheduling orders for multiple product types to minimize total weighted completion time. *Discrete Applied Mathematics* 155, 8 (2007), 945–970.

[25] Jinwei Liu and Haiying Shen. 2016. Dependency-aware and resource-efficient scheduling for heterogeneous jobs in clouds. In *2016 IEEE International Conference on Cloud Computing Technology and Science (CloudCom)*. 110–117.

[26] Gurobi Optimization LLC. 2018. Gurobi Optimizer. http://www.gurobi.com

[27] Monaldo Mastrolilli, Maurice Queyranne, Andreas S Schulz, Ola Svensson, and Nelson A Uhan. 2010. Minimizing the sum of weighted completion times in a concurrent open shop. *Operations Research Letters* 38, 5 (2010), 390–395.

[28] Vlad Nitu, Aram Kocharyan, Hannas Yaya, Alain Tchana, Daniel Hagimont, and Hrachya Astsatryan. 2018. Working Set Size Estimation Techniques in Virtualized Environments: One Size Does not Fit All. *Proceedings of the ACM on Measurement and Analysis of Computing Systems* 2, 1 (2018), 19.

[29] Zhen Qiu, Cliff Stein, and Yuan Zhong. 2015. Minimizing the total weighted completion time of coflows in datacenter networks. In *Proceedings of the 27th ACM symposium on Parallelism in Algorithms and Architectures*. ACM, 294–303.

[30] Maurice Queyranne and Maxim Sviridenko. 2002. A (2+ ε)-approximation algorithm for the generalized preemptive open shop problem with minsum objective. *Journal of Algorithms* 45, 2 (2002), 202–212.

[31] Jeff Rasley, Konstantinos Karanasos, Srikanth Kandula, Rodrigo Fonseca, Milan Vojnovic, and Sriram Rao. 2016. Efficient queue management for cluster scheduling. In *Proceedings of the 11th European Conference on Computer Systems*. 36.

[32] Jan Remy. 2004. Resource constrained scheduling on multiple machines. *Inform. Process. Lett.* 91, 4 (2004), 177–182.

[33] Sushant Sachdeva and Rishi Saket. 2013. Optimal inapproximability for scheduling problems via structural hardness for hypergraph vertex cover. In *Computational Complexity (CCC), 2013 IEEE Conference on*. IEEE, 219–229.

[34] Edward R Scheinerman and Daniel H Ullman. 2011. *Fractional graph theory: a rational approach to the theory of graphs*. Courier Corporation.

[35] Andreas S Schulz and Martin Skutella. 1997. Random-based scheduling new approximations and LP lower bounds. In *International Workshop on Randomization and Approximation Techniques in Computer Science*. Springer, 119–133.

[36] Malte Schwarzkopf, Andy Konwinski, Michael Abd-El-Malek, and John Wilkes. 2013. Omega: flexible, scalable schedulers for large compute clusters. In *Proceedings of the 8th ACM European Conference on Computer Systems*. ACM, 351–364.

[37] Uwe Schwiegelshohn. 2004. Preemptive weighted completion time scheduling of parallel jobs. *SIAM J. Comput.* 33, 6 (2004), 1280–1308.

[38] Mehrnoosh Shafiee and Javad Ghaderi. 2018. An improved bound for minimizing the total weighted completion time of coflows in datacenters. *IEEE/ACM Transactions on Networking* 26, 4 (2018), 1674–1687.

[39] Abhishek Verma, Luis Pedrosa, Madhukar Korupolu, David Oppenheimer, Eric Tune, and John Wilkes. 2015. Large-scale cluster management at Google with Borg. In *Proceedings of the 10th European Conference on Computer Systems*. 18.

[40] Weina Wang, Kai Zhu, Lei Ying, Jian Tan, and Li Zhang. 2016. Maptask scheduling in mapreduce with data locality: Throughput and heavy-traffic optimality. *IEEE/ACM Transactions on Networking (TON)* 24, 1 (2016), 190–203.

[41] John Wilkes. 2011. More Google cluster data. Google research blog. Posted at http://googleresearch.blogspot.com/2011/11/more-google-cluster-data.html.

[42] Ali Yekkehkhany, Avesta Hojjati, and Mohammad H Hajiesmaili. 2018. GB-PANDAS:: Throughput and heavy-traffic optimality analysis for affinity scheduling. *ACM SIGMETRICS Performance Evaluation Review* 45, 3 (2018), 2–14.

[43] Matei Zaharia, Dhruba Borthakur, Joydeep Sen Sarma, Khaled Elmeleegy, Scott Shenker, and Ion Stoica. 2010. Delay scheduling: a simple technique for achieving locality and fairness in cluster scheduling. In *Proceedings of the 5th European conference on Computer systems*. ACM, 265–278.

[44] Matei Zaharia, Andy Konwinski, Anthony D Joseph, Randy H Katz, and Ion Stoica. 2008. Improving MapReduce performance in heterogeneous environments. In *Osdi*, Vol. 8. 7.

[45] Matei Zaharia, Reynold S Xin, Patrick Wendell, Tathagata Das, Michael Armbrust, Ankur Dave, Xiangrui Meng, Josh Rosen, Shivaram Venkataraman, Michael J Franklin, et al. 2016. Apache spark: a unified engine for big data processing. *Commun. ACM* 59, 11 (2016), 56–65.

APPENDIX

A COMPLEXITY OF ALGORITHMS

The linear program (LP1) in (9) has at most $KNML + NL + N$ variables (K is the maximum number of tasks a job has.), which is polynomially bounded in the problem's input size. The number of constraints is also polynomially bounded. Hence, it can be solved in polynomial time using efficient linear programming solvers. The complexity of SynchPack-1 is mainly determined by solving (LP1). The complexity of Slow-Motion step is very low and can be parallelized in different machines, namely, $O(KNL)$ on each machine, and $O(KNLM)$ in total. The complexity of the greedy list scheduling – upon arrival or departure of a task fraction– is at most the length of the list (equal to the number of incomplete task fractions which is initially equal to $O(KNLM)$) times the number of machines M.

Mapping procedure is the extra step for SynchPack-2. The complexity of this step is also polynomially bounded in input size and is $O(K^2N^2ML)$. $O(KN + ML)$ is used for constructing the graph

as there are $O(KN)$ nodes on one side (number of all the tasks), $O(KN + ML)$ on the other side (number of all machine-interval copies), and it takes $O(KNML)$ to create edges (each task has at most 2 edges to copies of each machine-interval.). Further, finding an integral matching from the fractional matching takes $O(K^2N^2ML)$. The greedy algorithm in SynchPack-2 can be parallelized on the machines and takes $O(KN)$ in total.

Similarly, the complexity of SynchPack-3 is mainly dominated by solving (LP3) to find an appropriate ordering of jobs. The relaxed linear program (LP3) has $O(N^2)$ variables and $O(N^2 + MN)$ constraints and can be solved in polynomial time using efficient linear programming solvers. Note that the job ordering is the same on all the machines and they simply list-schedule their tasks respecting this ordering, independently of other machines. The complexity of the list scheduling is less than the one used in SynchPack-2 and is at most the length of the list, which is equal to the number of incomplete tasks.

Further, we would like to emphasize that in all the algorithms the corresponding linear program (LP) is solved *only once* at the beginning of the algorithm.

For the simulations, we used Gurobi software [26] to solve (LP2) and (LP3) in the simulations. On a desktop PC, with 8 Intel CPU core $i7-4790$ processors @ 3.60 GHz and 32.00 GB RAM, the average time it took to solve (LP1) was 145 seconds under offline setting. For purpose of comparison, the maximum job completion and the weighted average completion time time under our algorithm are 4.3×10^4 seconds and 8.6×10^3 seconds, respectively, for the case of priority-based weights. For solving (LP3), the average time it took was 435 seconds under offline setting, while the maximum job completion time and the weighted average completion time under our algorithm are 4.8×10^4 seconds and 10^4 seconds, respectively for the case of priority-based weights for $\alpha = 2$. We note that solving the LPs can be done much faster using the powerful computing resources in today's datacenters.

B PROOFS RELATED TO SYNCHPACK-1

B.1 Proof of Lemma 3.2

Consider an optimal solution to the task scheduling problem with packing and synchronization constraints. Define $\hat{C}^\star_{kj}$ (similarly, $\hat{C}^\star_j$) to be the left point of the interval in which task (k, j) (similarly, job j) completes in the optimal schedule. Clearly, $\hat{C}^\star_j \le C^\star_j$. We set $z^{il\star}_{kj}$ equal to the fraction of task (k, j) that is scheduled in interval l on machine i. Also, we set $x^\star_{j,l}$ to be one for the last interval that some task of job j is running in the optimal schedule and to be zero for other intervals. Obviously, $\hat{C}^\star_j = \sum_{l=0}^{L} d_{l-1} x^\star_{jl}$. It is easy to see that the set of values $\hat{C}^\star_j$, $z^{il\star}_{kj}$, and $x^\star_{j,l}$ satisfies all the constraints of (LP3). Therefore, $\sum_{j=1}^{N} w_j \tilde{C}_j \le \sum_{j=1}^{N} w_j \hat{C}^\star_j \le \sum_{j=1}^{N} w_j C^\star_j$.

B.2 Proof of Lemma 3.5

Recall that τ_l is the time that all the task fractions (k, j, i, l'), for $l' \le l$, complete in schedule $\mathcal{S}$. Let α_l be the fraction of job j that is completed by τ_l.

Note that as we schedule all the task fractions (k, j, i, l'), for $l' \le l$ and possibly some other task fractions, we have,

$$\alpha_l \ge \sum_{l'=0}^{l} \tilde{x}_{jl'}. \tag{23}$$

We define $y_{jl} = \alpha_l - \alpha_{l-1}$. Note that $\sum_{l=0}^{L} y_{jl} = 1$. Moreover, $C_j(\alpha) \le 3d_l$ for $\alpha \in (\alpha_{l-1}, \alpha_l]$. The factor 3 comes from Lemma 3.3. Therefore:

$$\begin{aligned}\int_0^1 C_j(\alpha)d\alpha &= \sum_{l=0}^{L} \int_{\alpha_{l-1}}^{\alpha_l} C_j(\alpha)d\alpha \le \sum_{l=0}^{L} (\alpha_l - \alpha_{l-1}) \times 3d_l \\ &\overset{(a)}{=} 3(1+\epsilon) \sum_{l=0}^{L} y_{jl} d_{l-1} \overset{(b)}{\le} 3(1+\epsilon) \sum_{l=0}^{L} \tilde{x}_{jl} d_{l-1} \\ &\overset{(c)}{=} 3(1+\epsilon)\tilde{C}_j,\end{aligned} \tag{24}$$

where (a) follows from definitions. Inequality (b) follows from (23) when y_{jl} and x_{jl} is seen as probabilities. Equality (c) comes from (9g) in (LP1).

B.3 Proof of Lemma 3.6

It is easy to observe that for every job j, $\tilde{C}^\lambda_j \le C_j(\lambda)/\lambda$. The reason is that $C_j(\lambda)$ is the time that λ fraction of job j is completed in $\mathcal{S}$; therefore, in the stretched schedule $\tilde{\mathcal{S}}$ by factor $1/\lambda$, job j is completed by time $C_j(\lambda)/\lambda$. Hence, we have

$$\begin{aligned}\mathbb{E}\left[\tilde{C}^\lambda_j\right] \le \mathbb{E}\left[C_j(\lambda)/\lambda\right] &\overset{(a)}{=} \int_0^1 \frac{C_j(\lambda)}{\lambda} \times 2\lambda \times d\lambda \\ &\overset{(b)}{\le} 6(1+\epsilon)\tilde{C}_j,\end{aligned}$$

where Equality (a) is by definition of expectation with respect to λ, with pdf $f(\lambda) = 2\lambda$, and Equality (b) is due to Lemma 3.5.

C DE-RANDOMIZATION

In this section, we discuss how to de-randomize the random choice of $\lambda \in (0, 1]$ in SynchPack-1, which was used to construct schedule $\tilde{\mathcal{S}}$ from schedule $\mathcal{S}$.

Recall that from Definition 3.4, $C_j(\lambda)$, $0 < \lambda \le 1$, is the starting point of the earliest interval in which λ-fraction of job j has been completed in schedule $\mathcal{S}$, which means at least λ-fraction of each of its tasks has been completed. We first aim to show that we can find

$$\lambda^\star = \arg\min_{\lambda \in (0,1]} \sum_{j \in \mathcal{J}} w_j C_j(\lambda)/\lambda \tag{25}$$

in polynomial time. Note that using the greedy packing algorithm, we schedule task fractions preemptively to form schedule $\mathcal{S}$. It is easy to see that $C_j(\lambda)$ is a step function with at most $O(L)$ breakpoints, since $C_j(\lambda) = d_l$ for some l and can get at most L different values. Consequently, $F(\lambda) = \sum_{j \in \mathcal{J}} w_j C_j(\lambda)$ is a step function with at most $O(NL)$ breakpoints. Let B denote the set of breakpoints of $F(\lambda)$. Thus, $F(\lambda)/\lambda = \sum_{j \in \mathcal{J}} w_j C_j(\lambda)/\lambda$ is a non-increasing function in intervals $(b, b']$, for b, b' being consecutive points in set B. This implies that,

$$\min_{\lambda \in (0,1]} F(\lambda)/\lambda = \min_{\lambda \in (0,1]} \sum_{j \in \mathcal{J}} w_j C_j(\lambda)/\lambda = \min_{\lambda \in B} \sum_{j \in \mathcal{J}} w_j C_j(\lambda)/\lambda.$$

We then can conclude that we can find $\lambda^\star$ in polynomial time by checking values of function $F(\lambda)/\lambda$ in at most $O(NL)$ points of set B and pick the one which incurs the minimum value. Given that, we have

$$\begin{aligned}\sum_{j\in\mathcal{J}} w_j \bar{C}_j^{\lambda^\star} &\le \sum_{j\in\mathcal{J}} (1+\epsilon) w_j C_j(\lambda^\star)/\lambda^\star \\ &\overset{(a)}{\le} (1+\epsilon)\mathbb{E}\Big[\sum_{j\in\mathcal{J}} w_j C_j(\lambda)/\lambda\Big] \\ &= (1+\epsilon)\sum_{j\in\mathcal{J}} w_j \int_{\lambda=0}^{1} \frac{C_j(\lambda)}{\lambda} 2\lambda d\lambda \\ &\overset{(b)}{=} 6(1+\epsilon)\sum_{j\in\mathcal{J}} w_j \tilde{C}_j,\end{aligned} \tag{26}$$

where (a) follows from (25). Equality (b) is due to Lemma 3.5. By choosing $\lambda = \lambda^\star$ in SynchPack-1, we have a deterministic algorithm with performance guarantee of $(6+\epsilon)\times$ OPT. , as stated by the following proposition.

D PROOFS RELATED TO SynchPack-2

D.1 Proof of Lemma 4.3

Consider an optimal solution to the non-preemptive task scheduling problem with packing and synchronization constraints. For each task, we set $z_{kj}^{il\,\star} = 1$ for the machine i and interval l if that task (k,j) is processed on i and finishes before d_l, and 0 otherwise. The rest of argument is similar to the proof of Lemma 3.2.

D.2 Proof of corollary 4.5

Note that (LP2) includes all the Constraints (9f)–(9h) of (LP1). Let α_l be the fraction of job j that is completed by interval l. Therefore,

$$\alpha_l = \sum_{l'=0}^{l} \tilde{x}_{jl'}. \tag{27}$$

Similar to Equations (24), we can write

$$\int_0^1 \hat{C}_j(\alpha)d\alpha = \sum_{l=0}^{L}(\alpha_l - \alpha_{l-1})\times d_{l-1} = \sum_{l=0}^{L}\tilde{x}_{jl} d_{l-1} = \tilde{C}_j,$$

D.3 Proof of Lemma 4.6

We use the following fundamental theorem (Theorem 2.1.3 in [34]): If there exists a fractional matching of some value v in a bipartite graph G, then there exists an integral matching of the same value v in G on the non-zero edges and can be found in polynomial time.

In our constructed bipartite graph $\mathcal{G}$, edge weights w_{kj}^{ilc} can be seen as a fractional matching. This is because for any node $u \in U$, the sum of weights of edges that are incident to u is 1, while for any node $v \in V$ the sum of weights of edges that are incident to v is at most 1. Recall that $|\cup_{j\in\mathcal{J}}\mathcal{K}_j| = \sum_{j\in\mathcal{J}}\sum_{k\in\mathcal{K}_j}\sum_{l=0}^{L}\bar{z}_{kj}^{il}$ is the number of total tasks. Setting $G = \mathcal{G}$ and $v = |\cup_{j\in\mathcal{J}}\mathcal{K}_j|$, an integral matching of nodes in U to nodes in V on non-zero edges can be found in polynomial time by the stated theorem.

D.4 Proof of Lemma 4.7

We now present the proof of Lemma 4.7 which bounds V_{il} (the total volume of tasks matched to all copies of interval l for machine i) by the product of the capacity of machine i and the length of interval l. Observe that due to definition of v_{kj}^i and Constraint (9d) we have,

$$\sum_{j\in\mathcal{J}}\sum_{k\in\mathcal{K}_j} v_{kj}^i \bar{z}_{kj}^{il} \le \bar{d}_l m_i, \tag{28}$$

The proof idea is similar to [23] that uses a simpler version of the mapping procedure in makespan minimization problem for scheduling tasks with unit resource requirements on unrelated machines with unit capacities, where each task can be scheduled in any machine. Let V_{il}^c denote the volume of the task that is matched to copy c of interval l on machine i. Thus, V_{il} is equal to the sum of V_{il}^c for all copies. Recall that we have $\lceil\bar{z}^{il}\rceil = \lceil\sum_{j\in J}\sum_{k\in\mathcal{K}_j}\bar{z}_{kj}^{il}\rceil$ many copies of interval l. Let $V_{il}^{\max}$ denote the largest volume of the task that is mapped to interval l. For this task, we know that $\bar{z}_{kj}^{il} > 0$ because the integral matching was found on nonzero edges (line 23 in Algorithm 4); hence, $p_{kj}^i \le d_l = \lambda\bar{d}_l \le \bar{d}_l$ by Constraint (13b) and $\lambda \in (0,1]$. In addition, let $v_{il}^{\min_c}$ denote the volume of the smallest task that has an edge with non-zero weight to copy c of interval l in graph $\mathcal{G}$ (or equivalently, has a non-zero edge in the fractional matching.). Observe that, the volume of the task that is matched to copy $c+1$ is at most $v_{il}^{\min_c}$. This is because of the way we construct graph $\mathcal{G}$ by sorting tasks according to their volumes for each machine (see the ordering in (16)) and the way we assign weights to edges. Thus,

$$\begin{aligned}V_{il} &= \sum_{c=1}^{\lceil\bar{z}^{il}\rceil} V_{il}^c \le V_{il}^{\max} + \sum_{c=2}^{\lceil\bar{z}^{il}\rceil} v_{il}^{\min_{c-1}} \\ &\overset{(a)}{\le} \bar{d}_l m_i + \sum_{j\in\mathcal{J}}\sum_{k\in\mathcal{K}_j}\sum_{c=1}^{\lceil\bar{z}^{il}\rceil-1} v_{kj}^i w_{kj}^{ilc}.\end{aligned}$$

Inequality (a) comes from the fact that $\sum_{j\in J}\sum_{k\in\mathcal{K}_j} w_{kj}^{ilc} \le 1$ and convex combination of some numbers is greater than the minimum number among them (note that the only copy for which we might have $\sum_{j\in J}\sum_{k\in\mathcal{K}_j} w_{kj}^{ilc} < 1$ is the last copy which is not considered in the left hand side of Inequality (a)). Therefore, as the direct result of the way we constructed graph $\mathcal{G}$, we have

$$V_{il} \le \bar{d}_l m_i + \sum_{j\in\mathcal{J}}\sum_{k\in\mathcal{K}_j} v_{kj}^i \bar{z}_{kj}^{il}$$

D.5 Proof of Lemma 4.8

Lemma 4.8 ensures that we can accommodate all the task fractions mapped to machine-interval (i,l) within an interval with length twice $d_l + \sum_{j\in\mathcal{J}}\sum_{k\in\mathcal{K}_j} v_{kj}^i \bar{z}_{kj}^{il}/m_i$.

Similar to Definition 2.1, we define $h(t)$ to be the height of the machine at time t. Assume that completion time of the last task, τ, is larger than $2V = 2\max(1,v)$, then

$$\sum_{j\in J} a_j p_j = \int_0^\tau h(t)dt > \int_0^{2V} h(t)dt \ge \int_0^V (h(t)+h(t+1))dt > 1+v,$$

where we have used the fact that $h(t) + h(t+1) > 1$, because otherwise the greedy scheduling can move tasks from time $t+1$ to time t as the greedy scheduling is non-preemptive and $p_j \leq 1$ for all tasks. Hence we arrived at a contradiction and the statement of Lemma 4.8 indeed holds.

D.6 Proof of Lemma 4.9

Let $\max(1, v) = 1$. We show correctness of Lemma 4.9 by constructing an instance for which an interval of size at least $2 - \zeta$ is needed to be able to schedule all the tasks for any $\zeta > 0$. Given a $\zeta > 0$, consider $n > log_2(1/\zeta) + 1$ tasks with processing times $1, 1/2, 1/4, \ldots, 1/2^{(n-1)}$ and size $1/2 + \eta$, for some $\eta > 0$ which is specified shortly. Note that we cannot place more than one of such tasks at a time on the machine, and therefore we need an interval of length $1 + 1/2 + 1/4 + \cdots + 1/2^{(n-1)} = 2 - 1/2^{(n-1)} > 2 - \zeta$ to schedule all the tasks. The total volume of tasks is equal to $(1/2 + \eta)(2 - 1/2^{(n-1)})$ which is less than 1, by choosing $\eta \leq 1/(2^{(n+1)} - 2)$. Therefore, for any $\zeta > 0$, we can construct an example for which an interval of length at least $2 - \zeta$ is needed to schedule all the tasks.

E PROOFS RELATED TO SynchPack-3

This section is devoted to the proof of the Theorem 5.2. We first characterize the solution of the linear program (LP3).

Lemma E.1. *Let $\tilde{C}_j$ be the optimal solution to (LP3) for completion time of job j, as in the ordering (22). For each machine i and each job j, $m_i \tilde{C}_j \geq \frac{1}{2} \sum_{k=1}^{j} v_{ik}$.*

Proof. Using Constraint (21b), for any machine $i \in \mathcal{M}$, we have

$$v_{ij} m_i \tilde{C}_j \geq v_{ij}^2 + \sum_{j' \in \mathcal{J}, j' \neq j} v_{ij} v_{ij'} \delta_{j'j}.$$

Hence, by defining $\delta_{kk} = 0$, it follows that

$$\sum_{k=1}^{j} v_{ik} m_i \tilde{C}_k \geq \frac{1}{2} \left(2 \sum_{k=1}^{j} v_{ik}^2 + \sum_{k=1}^{j} \sum_{k'=1}^{j} \left(v_{ik} v_{ik'} \delta_{k'k} + v_{ik} v_{ik'} \delta_{kk'} \right) \right) \tag{29}$$

We simplify the right-hand side of (29), using Constraint (21d), combined with the following equality

$$\sum_{k=1}^{j} v_{ik}^2 + \sum_{k=1}^{j} \sum_{\substack{k'=1 \\ k' \neq k}}^{j} v_{ik} v_{ik'} = (\sum_{k=1}^{j} v_{ik})^2,$$

and get

$$\sum_{k=1}^{j} v_{ik} m_i \tilde{C}_k \geq \frac{1}{2} \sum_{k=1}^{j} (v_{ik})^2 + \frac{1}{2} (\sum_{k=1}^{j} v_{ik})^2 \geq \frac{1}{2} (\sum_{k=1}^{j} v_{ik})^2. \tag{30}$$

Given that $\tilde{C}_j \geq \tilde{C}_k$ for $1 \leq k \leq j$, we get the final result. □

Let $C_j^\star$ be the completion time of job j in an optimal schedule, and $\text{OPT} = \sum_{j=1}^{N} w_j C_j^\star$ be the optimal value of our job scheduling problem. The following lemma states that the optimal value of (LP3), i.e., $\sum_{j=1}^{N} w_j \tilde{C}_j$, is a lower bound on the optimal value OPT.

Lemma E.2. $\sum_{j=1}^{N} w_j \tilde{C}_j \leq \sum_{j=1}^{N} w_j C_j^\star = OPT$.

Proof. Consider an optimal preemptive solution to the task scheduling problem with packing and synchronization constraints. We set the ordering variables such that $\delta_{jj'} = 1$ if job j precedes job j' in this solution, and $\delta_{jj'} = 0$, otherwise. We note that this set of ordering variables and job completion times satisfies Constraint (21b) since this solution will respect resource constraints on the machines. It also satisfies Constraint (21c). Therefore, the optimal solution can be converted to a feasible solution to (LP1). This implies the desired inequality. □

Let C_{ij} and C_j denote the completion time of task (i, j) and the completion time of job j under SynchPack-3, respectively. In the next step for the proof of Theorem 5.2, we aim to bound the total volume of the first j jobs (according to ordering (22)) that are processed during the time interval $(0, 4\tilde{C}_j]$ and subsequently use this result to bound C_j. *Note that the list scheduling policy used in* SynchPack-3 *is similar to the one used in* SynchPack-1, without the extra consideration for placement of fractions corresponding to the same task on different machines. Thus, The arguments here are similar to the ones in Lemma 3.3. Nevertheless, we present them for completeness.

Let T_{ij} denote the first time that all the first j tasks complete under SynchPack-3 on machine i. Recall that, as a result of Constraint (21c) and ordering in (22), $\tilde{C}_j \geq \tilde{C}_k \geq p_{ik}$ for all $k \leq j$ and all $i \in \mathcal{M}$. Further, the height of machine i at time t restricted to the first j jobs is denoted by $h_{ij}(t)$ and defined as the height of machine i at time t when only considering the first j jobs according to the ordering (22). We have the following lemma.

Lemma E.3. *Consider any interval $(\mathcal{T}_1, \mathcal{T}_2]$ for which $\mathcal{T}_2 - \mathcal{T}_1 = 2\tilde{C}_j$ and suppose $\mathcal{T}_2 < T_{ij}$ for some machine i. Then*

$$\sum_{t=\mathcal{T}_1+1}^{\mathcal{T}_2} h_{ij}(t) > m_i \tilde{C}_j \tag{31}$$

Proof of Lemma E.3. Without loss of generality, consider interval $(0, 2\tilde{C}_j]$ and assume $T_{ij} > 2\tilde{C}_j$. Let $S_{ij}(\tau)$ denote the set of tasks (i, k), $k \leq j$ (according to ordering (22)), running at time τ on machine i. We construct a bipartite graph $G = (U \cup V, E)$ as follows. For each time slot $\tau \in \{1, \ldots, 2\tilde{C}_j\}$ we consider a node z_τ, and define $U = \{z_\tau | 1 \leq \tau \leq \tilde{C}_j\}$, and $V = \{z_\tau | \tilde{C}_j + 1 \leq \tau \leq 2\tilde{C}_j\}$. For any $z_s \in U$ and $z_t \in V$, we add an edge (z_s, z_t) if $S_{ij}(t) \setminus S_{ij}(s) \neq \emptyset$, i.e., there is a task (i, k), $k \leq j$, running at time t that is not running at time s. Note that existence of edge (z_s, z_t) implies that $h_{ij}(s) + h_{ij}(t) > m_i$, because otherwise SynchPack-3 would have scheduled the task(s) in $S_{ij}(t) \setminus S_{ij}(s)$ (those that are running at t but not at s) at time s.

Next, we show that a perfect matching of nodes in U to nodes in V always exists in G. The existence of perfect matching then implies that any time slot $s \in (0, \tilde{C}_j]$ can be matched to a time slot $t \in (\tilde{C}_j, 2\tilde{C}_j]$ (one to one matching) and $h_{ij}(s) + h_{ij}(t) > m_i$. To prove that such a perfect matching always exists, we use Hall's Theorem [20]. For any set of nodes $\tilde{U} \subseteq U$, we define set of its neighbor nodes as $N_{\tilde{U}} = \{z_t \in V | \exists\, z_s \in \tilde{U} : (z_s, z_t) \in E\}$. Hall's Theorem states that a perfect matching exists if and only if for any $\tilde{U} \subseteq U$ we have $|\tilde{U}| \leq |N_{\tilde{U}}|$, where $|\cdot|$ denotes set cardinality (size). To arrive at a contradiction, suppose there is a (non-empty) set of nodes $\tilde{U} \subseteq U$ such that $|\tilde{U}| > |N_{\tilde{U}}|$. This implies that for a node z_{t_1}

in V but not in the neighbor set of $\tilde{U}$, i.e., $z_{t_1} \in V \setminus N_{\tilde{U}}$, we should have

$$S_{ij}(t_1) \setminus S_{ij}(s) = \varnothing, \tag{32}$$

for all $s, z_s \in \tilde{U}$. We now consider two cases:

Case (i): $|V \setminus N_{\tilde{U}}| = 1$, which means $|N_{\tilde{U}}| = \tilde{C}_j - 1$. But we had assumed $|\tilde{U}| > |N_{\tilde{U}}|$, thus $|\tilde{U}| = \tilde{C}_j$ and $\tilde{U} = U$. This implies that the tasks that are running at time t_1, are also running in the entire interval $(0, \tilde{C}_j]$; therefore, the processing time of each of them is at least $\tilde{C}_j + 1$ which contradicts the fact that $\tilde{C}_j \geq p_{ik}$ for all jobs $k \leq j$, by Constraint (21c) and ordering in (22).

Case (ii): $|V \setminus N_{\tilde{U}}| > 1$. In addition to the previous node z_{t_1}, consider another node $z_{t_2} \in V \setminus N_{\tilde{U}}$, and without loss of generality, assume $t_1 < t_2$. Similarly to (32), it holds that

$$S_{ij}(t_2) \setminus S_{ij}(s) = \varnothing, \tag{33}$$

for all $s, z_s \in \tilde{U}$. We claim that $S_{ij}(t_2) \subseteq S_{ij}(t_1)$, otherwise SynchPack-3 would have moved some task (i,k) running at t_2 and not at t_1 to time t_1 without violating machine i's capacity. This is feasible because, in view of (32) and (33), $(S_{ij}(t_1) \cup (i,k)) \setminus S_{ij}(s) = \varnothing$ for all $s, z_s \in \tilde{U}$. This implies that SynchPack-3 has scheduled all tasks of the set $S_{ij}(t_1) \cup (i,k)$ simultaneously at some time slot $s \in (0, \tilde{C}_j]$, which in turn implies that adding task (i,k) to time t_1 is indeed feasible (the total resource requirement of the tasks won't exceed m_i). Repeating the same argument for the sequence of nodes z_{t_1}, $z_{t_2}, \ldots, z_{t_{|V \setminus N_{\tilde{U}}|}}$, where $t_1 < t_2 < \cdots < t_{|V \setminus N_{\tilde{U}}|}$, we conclude that there exists a task that is running at all the times t, $z_t \in V \setminus N_{\tilde{U}}$, and at all the times $s, z_s \in \tilde{U}$. Therefore, its processing time is at least $\tilde{C}_j - |N_{\tilde{U}}| + |\tilde{U}|$ which is greater than $\tilde{C}_j$ by our assumption of $|\tilde{U}| > |N_{\tilde{U}}|$. This is a contradiction with the fact that $p_{ik} \leq \tilde{C}_j$ for all $k \leq j$ by Constraint (21c) and ordering (22).

Hence, we conclude that conditions of Hall's Theorem hold and a perfect matching in the constructed graph exists. As we argued, if $z_s \in U$ is matched to $z_t \in V$, we have $h_{ij}(s) + h_{ij}(t) > m_i$. Hence it follows that $\sum_{t=1}^{2\tilde{C}_j} h_{ij}(t) > m_i \tilde{C}_j$. □

Now we are ready to complete the proof of Theorem 5.2 regarding the performance of SynchPack-3.

Proof of Theorem 5.2. Recall that C_{ij} and C_j denote completion time of task (i,j) and completion time of job j under SynchPack-3, respectively. Also, T_{ij} denotes the first time that all the first j tasks are completed under SynchPack-3 on machine i. Therefore, $C_{ij} \leq T_{ij}$, by definition.

Define i_j to be the machine for which $C_j = C_{i_j j}$. If $T_{ij} \leq 4\tilde{C}_j$ for all machines $i \in \mathcal{M}$ and all jobs $j \in \mathcal{J}$, we can then argue that $\sum_{j=1}^{N} w_j C_j \leq 4 \times \text{OPT}$, because

$$\sum_{j=1}^{N} w_j C_j = \sum_{j=1}^{N} w_j C_{i_j j} \leq \sum_{j=1}^{N} w_j T_{i_j j} \overset{(a)}{\leq} 4 \sum_{j=1}^{N} w_j \tilde{C}_j \overset{(b)}{\leq} 4 \sum_{j=1}^{N} w_j C_j^{\star},$$

where Inequality (a) follows from our assumption that $T_{ij} \leq 4\tilde{C}_j$, and Inequality (b) follows from Lemma E.2.

Now to arrive at a contradiction, suppose $T_{ij} > 4\tilde{C}_j$ for some machine i and job j. We then have,

$$\begin{aligned} \sum_{k=1}^{j} v_{ik} = \sum_{t=1}^{T_{ij}} h_{ij}(t) &\overset{(c)}{>} \sum_{t=1}^{2\tilde{C}_j} h_{ij}(t) + \sum_{t=1}^{2\tilde{C}_j} h_{ij}(t + 2\tilde{C}_j) \\ &\overset{(d)}{>} m_i \tilde{C}_j + m_i \tilde{C}_j = 2 m_i \tilde{C}_j, \end{aligned} \tag{34}$$

where Inequality (c) is due to the assumption that $T_{ij} > 4\tilde{C}_j$, and Inequality (d) follows by applying Lemma E.3 twice, once for interval $(0, 2\tilde{C}_j]$ and once for interval $(2\tilde{C}_j, 4\tilde{C}_j]$. But (34) contradicts Lemma E.1. Hence, $\sum_{j=1}^{N} w_j C_j \leq 4 \times \text{OPT}$. □

F PSEUDOCODES OF (4)-APPROXIMATION ALGORITHM

Algorithm 1 provides a pseudocode for SynchPack-3, our preemptive 4-approximation algorithm, described in Section 5. The algorithm is a simple list scheduling based on the ordering obtained from (LP3).

Algorithm 1 Preemptive Scheduling Algorithm SynchPack-3

Given a set of machines $\mathcal{M} = \{1, ..., M\}$, a set of jobs $\mathcal{J} = \{1, ..., N\}$, and weights $w_j, j \in \mathcal{J}$:

1: Solve (LP1) and denote its optimal solution by $\{\tilde{C}_j; j \in \mathcal{J}\}$.
2: Order and re-index jobs such that $\tilde{C}_1 \leq \tilde{C}_2 \leq \ldots \leq \tilde{C}_N$.
3: On each machine $i \in \mathcal{M}$, apply list scheduling as described below:
4: **while** There is some incomplete task, **do**
5: List the incomplete tasks respecting the ordering in line 2. Let Q be the total number of tasks in the list. Denote current time by t, and set $h_i(t) = 0$
6: **for** $q = 1$ to Q **do**
7: Denote the q-th task in the list by (i, j_q)
8: **if** $h_i(t) + a_{ij_q} \leq m_i$, **then**
9: Schedule task (i, j_q).
10: Update $h_i(t) \leftarrow h_i(t) + a_{ij_q}$.
11: **end if**
12: **end for**
13: Process the tasks that were scheduled in line 9 until a task completes.
14: **end while**

G PSEUDOCODES OF $(6+\epsilon)$-APPROXIMATION ALGORITHM

A pseudocode for our preemptive $(6+\epsilon)$-approximation algorithm SynchPack-1 described in Section 3 is given in Algorithm 2. Line 1 in Algorithm 2 corresponds to Step 1 in Section 3, lines 2-18 correspond to Step 2, construction of schedule $\mathcal{S}$, and lines 19-20 describe Slow-Motion and construction of schedule $\bar{\mathcal{S}}$ in Step 3.

Algorithm 2 Preemptive Scheduling Algorithm SynchPack-1

Given a set of machines $\mathcal{M} = \{1, ..., M\}$, a set of jobs $\mathcal{J} = \{1, ..., N\}$, and weights $w_j, j \in \mathcal{J}$:

1: Solve (LP1) and denote its optimal solution by $\{\tilde{z}^{il}_{kj};\ j \in \mathcal{J},\ k \in \mathcal{K}_j,\ i \in \mathcal{M},\ l \in \{0, 1, \ldots, L\}\}$.
2: List non-zero task fractions (i.e., tasks (k, j) with size a_{ij} and non-zero fractional duration $\tilde{z}^{il}_{kj} p^i_{kj}$) such that task fraction (k, j, i, l) appears before task fraction (k', j', i', l'), if $l < l'$. Task fractions within each interval ando corresponding to different machines are ordered arbitrarily.
3: **while** There is some unscheduled task fraction, **do**
4: List the unscheduled task fractions. Let Q be the total number of task fractions in the list and $l^\star$ be the interval with minimum value that has some unscheduled task fractions..
5: Denote current time by t, and set $h_i(t)$ to be the height of machine i at t.
6: **for** $q = 1$ to Q **do**
7: Denote the q-th task (whose fraction need to get scheduled) in the list by (k_q, j_q).
8: **if** $h_i(t) + a_{k_q j_q} \le m_i$ and no fraction of task (k_q, j_q) is running in any other machine, **then**
9: Schedule task (k_q, j_q) to run on machine i.
10: Update $h_i(t) \leftarrow h_i(t) + a_{k_q j_q}$.
11: **end if**
12: **end for**
13: Process the task fractions that were scheduled in line 12 until a task fraction completes.
14: Preempt scheduling of all the task fractions (k, j, i, l) with $l > l^\star$.
15: Update $\tilde{z}^{il}_{kj} \leftarrow \tilde{z}^{il}_{kj} - \tau / p^i_{kj}$, where τ is the amount of time it gets processed.
16: **if** $\tilde{z}^{il}_{kj} = 0$ **then**
17: Remove task (k_q, j_q) from the list of machine i, and update $Q \leftarrow Q - 1$.
18: **end if**
19: **end while**
20: Denote the obtained schedule by $\mathcal{S}$. Choose λ randomly from $(0, 1]$ with pdf $f(\lambda) = 2\lambda$.
21: Construct schedule $\bar{\mathcal{S}}$ by applying Slow-Motion with parameter λ to $\mathcal{S}$. Process jobs according to $\bar{\mathcal{S}}$.

H PSEUDOCODE OF 24-APPROXIMATION ALGORITHM

Algorithm 3 provides a pseudocode for our non-preemptive algorithm, SynchPack-2, described in Section 4. Line 1 in Algorithm 3 corresponds to Step 1 in SynchPack-2 and lines 2 corresponds to Step 2, namely, construction of preemptive schedule and applying Slow-Motion. Lines 3-11 describes the procedure of constructing a non-preemptive schedule using $\bar{\mathcal{S}}$ in Step 3.

Algorithm 4 describes the mapping procedure which is used as a subroutine in Algorithm 3.

Algorithm 3 Non-Preemptive Scheduling Algorithm SynchPack-2

Given a set of machines $\mathcal{M} = \{1, ..., M\}$, a set of jobs $\mathcal{J} = \{1, ..., N\}$, and weights $w_j, j \in \mathcal{J}$:

1: Solve (LP2) and denote its optimal solution by $\{\tilde{z}^{il}_{kj},\ j \in \mathcal{J},\ k \in \mathcal{K}_j,\ i \in \mathcal{M},\ 0 \le l \le L\}$.
2: Apply Slow-Motion by choosing λ randomly from $(0, 1]$ with pdf $f(\lambda) = 2\lambda$, and define $\bar{z}^{il}_{kj}$, as in (15).
3: Run Algorithm 4 and output list of tasks that are mapped to each machine-interval (i, l), $i \in \mathcal{M},\ l \le L$.
4: **for** Each machine $i \in \mathcal{M}$, **do**
5: **for** Each interval l, $0 \le l \le L$, **do**
6: List the unscheduled task fractions. Let Q be the total number of task fractions in the list.
7: **while** There is some unscheduled task fraction, **do**
8: Denote current time by t, and set $h_i(t)$ to be the height of machine i at t.
9: **for** $q = 1$ to Q **do**
10: Denote the q-th task in the list by (k_q, j_q)
11: **if** $h_i(t) + a_{k_q j_q} \le m_i$, **then**
12: Schedule task (k_q, j_q).
13: Update $h_i(t) \leftarrow h_i(t) + a_{k_q j_q}$.
14: Update $Q \leftarrow Q - 1$.
15: **end if**
16: **end for**
17: Process the task fractions that were scheduled in line 12 until a task fraction is complete.
18: **end while**
19: **end for**
20: **end for**

Algorithm 4 Procedure of Mapping Tasks to Intervals

Given a set of jobs $\mathcal{J} = \{1, ..., N\}$, with task volumes v^i_{kj} on machine i, and values of $\bar{z}^{il}_{kj}$:

Construct bipartite graph $\mathcal{G}_i = (U \cup V, E)$ as follows:
For each task (k, j), $j \in J,\ k \in \mathcal{K}_j$, add a node in U.
for Each machine i, $i \in \mathcal{M}$, **do**
    Order and re-index tasks such that: $v^i_{k_1 j_1} \geq v^i_{k_2 j_2} \geq \ldots v^i_{k_{N_i} j_{N_i}} > 0$.
    for Each interval l, $l \leq L$, **do**
        Consider $\lceil \bar{z}^{il} \rceil = \lceil \sum_{j \in J} \sum_{k \in \mathcal{K}_j} \bar{z}^{il}_{kj} \rceil$ consecutive nodes in V_i, and set $W^{ic_l}_l = 0$ for $1 \leq c_l \leq \lceil \bar{z}^{il} \rceil$. Also set $c_l = 1$.
        for $q = 1$ to N_i **do**
            $R = \bar{z}^{il}_{kj}$,
            while $R \neq 0$ **do**
                Add an edge between the node (k_q, j_q) in set U and node $c_l \in V_i$.
                Assign weight $w^{ilc}_{kj} = \min\{R, 1 - W^{c_l}_l\}$.
                Update $R \leftarrow R - w^{ilc}_{kj}$.
                Update $W^{c_l}_l \leftarrow W^{c_l}_l + w^{ilc}_{kj}$
                if $W^{c_l}_l = 1$ **then**
                    $c_l = c_l + 1$.
                end if
            end while
        end for
    end for
end for
Set $V = \cup_{i \in \mathcal{M}} V_i$.
Find an integral matching in $\mathcal{G}$ on the nonzero edges with value $|\cup_{j \in \mathcal{J}} \mathcal{K}_j| = \sum_{j \in \mathcal{J}} \sum_{k \in \mathcal{K}_j} \sum_{l=0}^{L} \bar{z}^{il}_{kj}$.